

## PROFIsafe Driver V2.2.3 for F-Slaves

Programming Manual

Preface

Product overview

**1**

Method of functioning of  
the PSD

**2**

Files of the PSD

**3**

Interface Description

**4**

Configuration of the PSD

**5**

Requirements

**6**

Test specification

**7**

Appendix

**A**

Index of abbreviations /  
terms

**B**

Bibliography

**C**

## Legal information

### Warning notice system

This manual contains notices you have to observe in order to ensure your personal safety, as well as to prevent damage to property. The notices referring to your personal safety are highlighted in the manual by a safety alert symbol, notices referring only to property damage have no safety alert symbol. These notices shown below are graded according to the degree of danger.

#### **DANGER**

indicates that death or severe personal injury **will** result if proper precautions are not taken.

#### **WARNING**

indicates that death or severe personal injury **may** result if proper precautions are not taken.

#### **CAUTION**

indicates that minor personal injury can result if proper precautions are not taken.

#### **NOTICE**

indicates that property damage can result if proper precautions are not taken.

If more than one degree of danger is present, the warning notice representing the highest degree of danger will be used. A notice warning of injury to persons with a safety alert symbol may also include a warning relating to property damage.

### Qualified Personnel

The product/system described in this documentation may be operated only by **personnel qualified** for the specific task in accordance with the relevant documentation, in particular its warning notices and safety instructions. Qualified personnel are those who, based on their training and experience, are capable of identifying risks and avoiding potential hazards when working with these products/systems.

### Proper use of Siemens products

Note the following:

#### **WARNING**

Siemens products may only be used for the applications described in the catalog and in the relevant technical documentation. If products and components from other manufacturers are used, these must be recommended or approved by Siemens. Proper transport, storage, installation, assembly, commissioning, operation and maintenance are required to ensure that the products operate safely and without any problems. The permissible ambient conditions must be complied with. The information in the relevant documentation must be observed.

### Trademarks

All names identified by ® are registered trademarks of Siemens AG. The remaining trademarks in this publication may be trademarks whose use by third parties for their own purposes could violate the rights of the owner.

### Disclaimer of Liability

We have reviewed the contents of this publication to ensure consistency with the hardware and software described. Since variance cannot be precluded entirely, we cannot guarantee full consistency. However, the information in this publication is reviewed regularly and any necessary corrections are included in subsequent editions.

# Preface

## Purpose of the Manual

The manual describes the procedure for integrating the "PROFIsafe Driver for F-Slaves" (hereinafter abbreviated as PSD) in various runtime environments. It refers to PSD V2.2.3 /1/, which has implemented the mechanism in accordance with the PROFIsafe V2.6 MU 1. The document can be obtained from the PROFIBUS User Organization (PNO) and is also available on the PROFIsafe Driver CD.

## Application

The PSD for F-Slaves is used in PROFIBUS and PROFINET applications with safety-related (fail-safe) field devices. These can, for example be: Remote I/Os, laser scanners, light curtains, drives, valves, robots, HMI devices, pressure transmitters, overfill protections, gateways to other fail-safe buses and many others. In this document, names for safety-related equipment are given the prefix "F" (Fail-safe), for example, F-slave and F-host.

## Changes compared with the previous edition PSD V2.2.2

- Support of 64-bit controller variants
- Multi-instance is possible for 8-bit controllers
- Implementation of unique error IDs (Page 84)

## Customer benefits

The certified PSD for slaves reduces development and certification work for F-device manufacturers. The development of the PSD through a PNO member provides the following advantages:

- The F-communication works the same way in all F-slaves that use the PSD (this ensures the PROFIsafe profile /1/).
- The work to implement, test and certify the PSD code only needs to be performed once.

## Conditions of use

## Data types

The PSD offers the following variables for the data types used:

- unsigned char: 8 bits
- unsigned short: 16 bits
- unsigned long: 32 bits
- unsigned long long: 64 bits (this data type is only used for activated generation parameters "WITH\_64BIT\_EMULATION" in p\_c\_config.h)

The PSD checks the size of the data types it uses and goes into the PSD\_HARD\_FAIL state when a deviation occurs.

## Byte order

A microcontroller with little or big endian byte order can be used. The PSD detects the two types of byte orders and adapts itself.

The PSD is not tested for 2-channel F-slaves, in which the two channels are operated with a different byte order. This must be taken into consideration for the synchronization of the F-application.

## C compiler

The PSD is designed according to the ANSI standard C90.

If the "Define WITH\_64BIT\_EMULATION" is commented out in p\_c\_config.h, it is in accordance with ANSI standard C99. The 64-bit data type "long long" first exists as of C99.

The following compilers were used:

- **AVR-GCC 7.0.895 ANSI-C Compiler for 8-bit AVR ATMEGA 2560** ®
- Microchip XC16 v1.33 for PIC16 MCU ®
- Microchip XC32 v2.10 for PIC32 MCU ®
- Green Hills Compiler for ARM V.2013.5.4 ®
- Microchip Cross Compiler for Raspberry Pi with 64 bit (**gcc-aarch64-linux-gnu gcc-linaro-6.3.1-2017.02**) ®

The PSD does not use any library functions.

## Use under C++

The PSD may also be used in C++ projects. The definition for C++ is installed in the header files of the interface functions (p\_api.h, p\_c\_fapplication.h and p\_c\_si.h) for this.

### NOTICE

Depending on the hardware used, the compiler used and the compiler settings selected, it is the system integrator's responsibility to verify correct functioning of the PSD by means of suitable verification measures.

The tested code should correspond to the release code as far as possible (e.g. with regard to the compiler settings used for debug options, code optimization, RunTimeCheck).

## Readership

Development departments of PROFIsafe devices.

## Guide

The manual contains the following aids that provide quick access to the specific information you need:

- You can find a complete table of contents at the beginning of the manual.
- The appendix includes a list of abbreviations and glossary of terms, in which important technical terms used in this manual are defined.
- References to other documents are indicated by means of literature numbers in slashes ("/no./"). This allows you refer to the exact title of the document from the bibliography in the appendix of the manual.

## Further Support

Find answers quickly and easily on the Internet

(<https://new.siemens.com/global/en/products/automation/industrial-communication/profinet/development/comdec.html>) and via the email address of ComDeC, ComDeC@siemens.com.

You can find additional information on the topic of PROFIBUS on the Internet

(<https://www.profibus.com>).

## Important note for maintaining the operational safety of your system

---

### Note

The operators of systems with safety-related characteristics must adhere to specific operational safety requirements. The supplier is also obliged to comply with special product monitoring measures. Siemens publishes a special newsletter to keep system operators informed about product developments and properties which may form important issues in terms of operational safety. You should subscribe to the corresponding newsletter in order to obtain the latest information and to allow you to modify your system accordingly. Go to the Internet (<https://support.industry.siemens.com/cs/ww/en>) and enter type the article number "6ES7195-3BF03-0YA0" in the "Search for product information" box. Then click in the search results right on "E-mail with update".

---

Siemens provides products and solutions with industrial security functions that support the secure operation of plants, systems, machines and networks.

In order to protect plants, systems, machines and networks against cyber threats, it is necessary to implement – and continuously maintain – a holistic, state-of-the-art industrial security concept. Siemens' products and solutions constitute one element of such a concept.

Customers are responsible for preventing unauthorized access to their plants, systems, machines and networks. Such systems, machines and components should only be connected to an enterprise network or the internet if and to the extent such a connection is necessary and only when appropriate security measures (e.g. firewalls and/or network segmentation) are in place.

For additional information on industrial security measures that may be implemented, please visit (<https://www.siemens.com/industrialsecurity>).

Siemens' products and solutions undergo continuous development to make them more secure. Siemens strongly recommends that product updates are applied as soon as they are available and that the latest product versions are used. Use of product versions that are no longer supported, and failure to apply the latest updates may increase customers' exposure to cyber threats.

To stay informed about product updates, subscribe to the Siemens Industrial Security RSS Feed visit (<https://www.siemens.com/industrialsecurity>).

# Table of contents

|          |                                                     |           |
|----------|-----------------------------------------------------|-----------|
|          | <b>Preface .....</b>                                | <b>3</b>  |
| <b>1</b> | <b>Product overview .....</b>                       | <b>10</b> |
| 1.1      | Description of the PROFIsafe driver (PSD) .....     | 10        |
| 1.2      | Safe communication between F-host and F-slave ..... | 10        |
| 1.3      | Fault detection mechanisms in the PSD .....         | 12        |
| 1.4      | Standards and guidelines .....                      | 12        |
| <b>2</b> | <b>Method of functioning of the PSD .....</b>       | <b>13</b> |
| 2.1      | Embedding of the PSD in F-slaves/F-submodules ..... | 13        |
| 2.1.1    | Tasks of the stack.....                             | 14        |
| 2.1.2    | Tasks of the stack interface .....                  | 15        |
| 2.1.3    | Tasks of the F-application:.....                    | 15        |
| 2.1.4    | The synchronization interface .....                 | 16        |
| 2.2      | States of the PSD .....                             | 17        |
| 2.2.1    | Description of the states .....                     | 18        |
| 2.2.2    | Description of the state transitions .....          | 19        |
| 2.2.3    | Activities of the PSD in the PSD_DATAEX state ..... | 20        |
| 2.2.3.1  | Processing of the output message frames.....        | 20        |
| 2.2.3.2  | Processing of the input message frames.....         | 23        |
| <b>3</b> | <b>Files of the PSD .....</b>                       | <b>25</b> |
| 3.1      | Files that cannot be changed .....                  | 25        |
| 3.2      | Files that can be changed.....                      | 28        |
| 3.3      | Commissioning procedure .....                       | 28        |
| <b>4</b> | <b>Interface Description .....</b>                  | <b>29</b> |
| 4.1      | PSD interface functions .....                       | 29        |
| 4.1.1    | psd_InitInstance(...).....                          | 30        |
| 4.1.2    | psd_FParBuild(...).....                             | 31        |
| 4.1.3    | psd_Config(...).....                                | 34        |
| 4.1.4    | psd_Run(...).....                                   | 36        |
| 4.1.5    | psd_Stop(...).....                                  | 37        |
| 4.1.6    | psd_RecvFOutTele(...).....                          | 38        |
| 4.1.7    | psd_RecvFOutTele_Invers(...) .....                  | 39        |
| 4.1.8    | psd_GetFOutData(...).....                           | 40        |
| 4.1.9    | psd_GetFOutData_Invers(...) .....                   | 44        |
| 4.1.10   | psd_SetFInData(...) .....                           | 47        |
| 4.1.11   | psd_SetFInData_Invers(...) .....                    | 48        |
| 4.1.12   | psd_SendFInTele(...) .....                          | 50        |
| 4.1.13   | psd_SendFInTele_Invers(...) .....                   | 51        |

|         |                                                                      |    |
|---------|----------------------------------------------------------------------|----|
| 4.2     | PSD interface variable psd_SafetyVar.....                            | 52 |
| 4.3     | Functions to be implemented by the F-application .....               | 52 |
| 4.3.1   | hard_err(...).....                                                   | 52 |
| 4.3.2   | psd_GetFParPtr(...).....                                             | 53 |
| 4.3.3   | psd_GetConfigPtr(...) .....                                          | 54 |
| 4.3.4   | psd_GetOutputTelegram(...) .....                                     | 54 |
| 4.3.5   | psd_GetInputBuffer(...).....                                         | 55 |
| 4.3.6   | psd_InputSendAck(...).....                                           | 55 |
| 4.3.7   | enable_int(...).....                                                 | 56 |
| 4.3.8   | disable_int(...) .....                                               | 56 |
| 4.4     | Functions to be implemented at the 2-channel generation variant..... | 57 |
| 4.4.1   | psd_OutTransfer(...) .....                                           | 57 |
| 4.4.2   | psd_OutSync(...).....                                                | 58 |
| 4.4.3   | psd_InTransfer(...) .....                                            | 59 |
| 4.4.4   | psd_InSync(...).....                                                 | 60 |
| 4.5     | Optional PSD interface functions.....                                | 61 |
| 4.5.1   | Auxiliary functions for the CRC calculation.....                     | 61 |
| 4.5.1.1 | psd_CRC16(...).....                                                  | 61 |
| 4.5.1.2 | psd_CRC24(...).....                                                  | 62 |
| 4.5.1.3 | psd_CRC32(...).....                                                  | 63 |
| 4.5.2   | psd_GetState(...) .....                                              | 63 |
| 4.5.3   | psd_GetFPar(...).....                                                | 64 |
| 4.5.4   | psd_GetConfig(...) .....                                             | 65 |
| 4.5.5   | psd_GetPSD_Version(...) .....                                        | 66 |
| 4.5.6   | psd_Falsify_InTelCRC(...).....                                       | 67 |
| 4.5.7   | psd_GetTraceData(...) .....                                          | 68 |
| 4.5.8   | Inverse PSD functions .....                                          | 69 |
| 4.5.8.1 | psd_InitInstance_Invers(...).....                                    | 69 |
| 4.5.8.2 | psd_FParBuild_Invers(...).....                                       | 70 |
| 4.5.8.3 | psd_Config_Invers(...).....                                          | 71 |
| 4.5.8.4 | psd_Run_Invers(...).....                                             | 71 |
| 4.5.8.5 | psd_Stop_Invers(...).....                                            | 72 |
| 4.5.8.6 | psd_GetFPar_Invers(...).....                                         | 72 |
| 4.5.8.7 | psd_GetConfig_Invers(...) .....                                      | 73 |
| 4.5.8.8 | psd_GetState_Invers(...).....                                        | 73 |

|          |                                                                                              |           |
|----------|----------------------------------------------------------------------------------------------|-----------|
| <b>5</b> | <b>Configuration of the PSD .....</b>                                                        | <b>74</b> |
| 5.1      | Number of PSD instances .....                                                                | 74        |
| 5.2      | Hardware architecture .....                                                                  | 74        |
| 5.3      | Instance addressing .....                                                                    | 75        |
| 5.4      | Checking of the source address .....                                                         | 75        |
| 5.5      | Support of the V1 communication mode .....                                                   | 75        |
| 5.6      | Selection for using 64 bit arithmetic functions provided compiler-internal or from PSD ..... | 76        |
| 5.7      | Integration of fault insertion tests in the PSD OP code .....                                | 76        |
| 5.8      | Consecutive No. for fault-insertion test activation.....                                     | 76        |
| 5.9      | F-input or F-output user data area .....                                                     | 77        |
| 5.10     | File ID as debug information .....                                                           | 77        |
| 5.11     | Memory attributes .....                                                                      | 78        |
| 5.12     | Memory model for the CRC tables .....                                                        | 78        |
| <b>6</b> | <b>Requirements .....</b>                                                                    | <b>79</b> |
| 6.1      | Requirements independent of the generation variable.....                                     | 79        |
| 6.2      | Requirements for the 1-channel generation variant .....                                      | 83        |
| 6.3      | Requirements for the 2-channel generation variant .....                                      | 83        |
| <b>7</b> | <b>Test specification .....</b>                                                              | <b>84</b> |
| 7.1      | Fault insertion test.....                                                                    | 84        |
| 7.1.1    | 1-channel and 2-channel generation variant .....                                             | 85        |
| 7.1.2    | 1-channel generation variant .....                                                           | 89        |
| 7.1.3    | 2-channel generation variant .....                                                           | 90        |
| 7.2      | Code generation .....                                                                        | 91        |
| 7.3      | Compiler verification code for the 1-channel generation variant .....                        | 91        |
| 7.4      | PROFIsafe Conformance-Test .....                                                             | 92        |
| <b>A</b> | <b>Appendix.....</b>                                                                         | <b>93</b> |
| A.1      | Assignment of return values to diagnostic messages .....                                     | 93        |
| A.2      | Required space .....                                                                         | 94        |
| A.2.1    | 1-channel generation (16-bit controller).....                                                | 94        |
| A.2.2    | 2-channel generation (16-bit controller).....                                                | 94        |
| A.3      | Static code analysis.....                                                                    | 94        |
| <b>B</b> | <b>Index of abbreviations / terms.....</b>                                                   | <b>95</b> |
| <b>C</b> | <b>Bibliography.....</b>                                                                     | <b>96</b> |

## Product overview

### 1.1 Description of the PROFIsafe driver (PSD)

The PSD is a firmware module for F-slaves/F-submodules. Here the monitoring mechanisms for safe communication are implemented as specified under the "PROFIsafe – Profile for Safety Technology on PROFIBUS and PROFINET (IEC61784-3-3)". Its use allows manufacturers of F-slaves simple connection of their F-slaves to the standard PROFIBUS as well as standard PROFINET and thus fail-safe communication with an F-communication partner (F-host).

Equipment manufacturers can use the PSD to develop an F-slave with fail-safe, point-to-point communication via PROFIBUS or PROFINET. The PSD can be integrated into various runtime environments (HW / FW environments) through "generation".

### 1.2 Safe communication between F-host and F-slave

The essential task of the PSD is to exchange user data with an F-error host and the resources for F-parameterization according to the specifications of the PROFIsafe profile /1/. The layout of a full input/output frame corresponds to the definition in the PROFIsafe profile (Fig. "Safety PDU for CPF3") /1/. The terms in accordance with the following figure are used below.

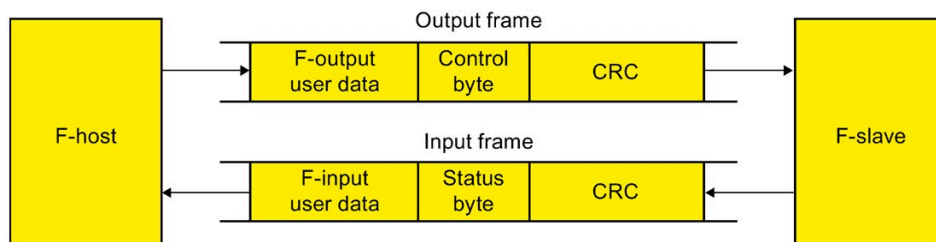


Figure 1-1 Structure of F-frames

The following data are relevant for the PSD user:

#### Output message frame:

- F-output user data  
The structure depends on the application.
- Control byte.Bit\_0: iPar\_EN  
The F-host gives the control information iPar\_EN about the F-slave to the F-application of the F-device.
- Control byte.Bit\_1: OA\_Req  
The F-slave can display an operator acknowledgment request.
- Control byte.Bit\_4: activate\_FV  
Used by the F-host to have the F-slave output failsafe values at its outputs.
- Control byte.Bit\_6: ChF\_Ack  
Set by the F-application in the F-host as acknowledgment to a ChF\_Ack\_Req from the F-slave.

**Input message frame:**

- F-input user data  
The structure depends on the application.
- Status byte.Bit\_0: iPar\_OK  
The F-application gives the status information iPar\_OK about the F-slave to the F-host.
- Status byte.Bit\_1: Device\_Fault or ChF\_Ack\_Req
  - With F-parameter "F\_Passivation" = 0 (module-wide passivation): DeviceFault  
The F-application has identified a safety-related malfunction.
  - With F-parameter "F\_Passivation" = 1 (channel granular passivation): ChF\_Ack\_Req  
The F-application has deleted at least one channel error.
- Status byte.Bit\_4: FV\_activated  
The F-slave uses fail-safe values at its outputs.

The meanings of the bits in the status/control byte are describe in the PROFIsafe profile (Fig. "Status byte" or "Control byte") /1/.

**WARNING****Requirement: PSD\_RQ\_0\_14**

The F-application must set the Status byte.Bit\_1 (Device\_Fault) for at least 2 changes of the ConsecutiveNumber (2 x the value "F\_OUTPUT\_OK\_NEW\_TEL" in Teleststatus\_2 of the PSD function psd\_GetFOutData (...)), if it is unable to respond within the configured timeout period.

## 1.3 Fault detection mechanisms in the PSD

The hardware components (RAM, microcontroller, timer, etc.) of the F-slave in which the PSD is integrated, are not known by the PSD. Therefore, no hardware tests such as RAM tests, command tests or timer tests are implemented in it. The objective of the F-application is to detect the hardware error corresponding to the desired F-slave SIL.

The mechanisms for fault detection from the PROFIsafe Profile /1/ are implemented in the PSD. The probability considerations (for example residual fault probability) are specified in the PROFIsafe Profile chapter "Constraints for the calculation of system characteristics".

Various measures for detecting the following faults are furthermore implemented in the PSD:

- Faulty program flow
- Incorrectly saved PSD variables
- Monitoring time for F-communications exceeded

The measures act immediately during the execution of the PSD code, so that an additional PSD self-test function that is called up cyclically is not required.

In the 1-channel as well as in 2-channel PSD, a quality program flow control is incorporated, for example, in all the interface functions, to detect falsification in the program flow.

In the 1-channel PSD the functions are processed logically in two channels, with the data being stored inverted in the second channel. This can also be seen in the names of the PSD interface functions in the following sections ("psd\_...\_Invers").

In a 2-channel PSD synchronization functions in cyclic operation are used to implement a process control. This means that the results of the program flow control of both channels are compared with each other.

## 1.4 Standards and guidelines

The PSD was developed in accordance with the SIL3 requirements of IEC 61508-1 and 3 /2/. The PSD fulfils the requirements of the standards and directives with regard to functional safety, as specified in the report on the safety certificate (TÜV certificate). The latest certification documents can be found on the Internet (<https://support.automation.siemens.com/WW/view/en/11669702/134200>) as soon as they are available.

Information to achieve an adequate security standard can be found in /4/.

## Method of functioning of the PSD

### 2.1 Embedding of the PSD in F-slaves/F-submodules

The following images show the FW blocks involved in the F-communication and the embedding of the PSD.

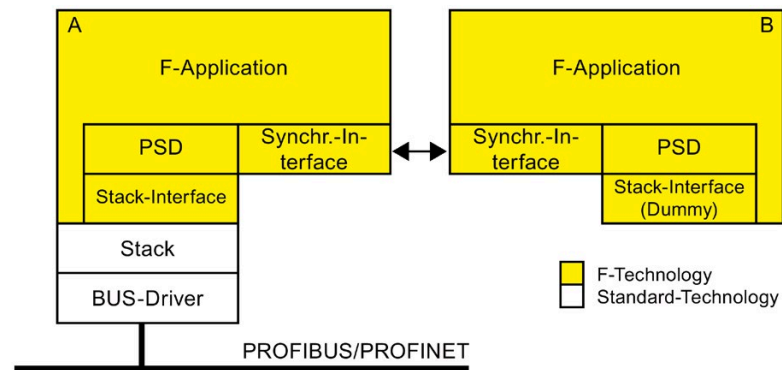


Figure 2-1 Embedding of the PSD (2-channel)

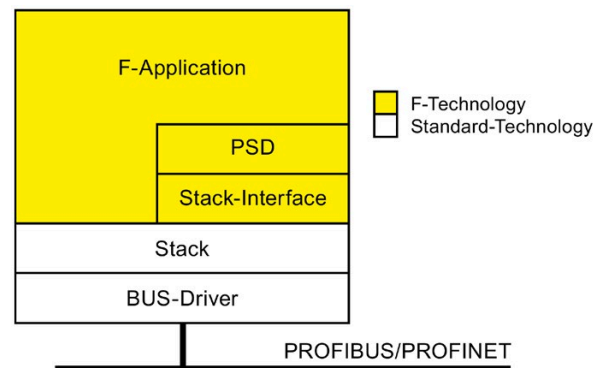


Figure 2-2 Embedding of the PSD (1-channel)



**WARNING**

**Requirement: PSD\_RQ\_0\_01**

The normative requirements for HW and FW development and for the implemented HW and FW must be have complied with in accordance with the underlying standards relating to the F-application!

The PSD knows no hardware components at all e.g. memory, microcontroller, timer, I/O interface, power supply. Therefore the PSD does not have any background tests, such as memory test, command test timer test, implemented.

The PSD has implemented measures to locate the transmission error, error in its program flow and corruption of its variables.

The selection of suitable measures for detecting random hardware errors depends on the system architecture used and lies in the responsibility of the F-slave development.

**Stack:** Controls communication via PROFIBUS or PROFINET.

**Stack interface:** Used to adapt the PSD to different stacks

**F-application:** Specific device application that the PSD uses for communication via PROFIBUS or PROFINET with cyclic F-frames.

**Synchronization interface:** for data exchange

The tasks of the FW blocks worded below serve the orientation when designing an F-slave.

### 2.1.1 Tasks of the stack

The following tasks are done by the stack:

- The stack receives F-parameters and optional iParameters (individual device parameters of the F-slave). The stack signals the receipt to the F-application.
- The stack receives the output frames and makes them available to the F-application consistently, i.e. coming from the same output frame.
- The stack generates the PROFIBUS/PROFINET input frames from the PROFIsafe frames provided by the PSD and sends them to the F-host.
- The stack detects events of the communication bus, such as "Start/Stop of cyclic data exchange" and makes them available to the F-application.

### 2.1.2 Tasks of the stack interface

The following tasks are done by the stack interface:

- The stack interface extracts the F-parameters from the parameter frame of the stack and makes them available to the PSD in a buffer.
- The stack interface takes the output frames from the stack and makes them available to the PSD in a buffer.
- The stack interface passes a buffer for applying the input message frame to the PSD on request respectively.
- The stack interface accepts the buffer with the input message frame.
- The stack interface regulates the sending of the input frame to the F-host with the stack.

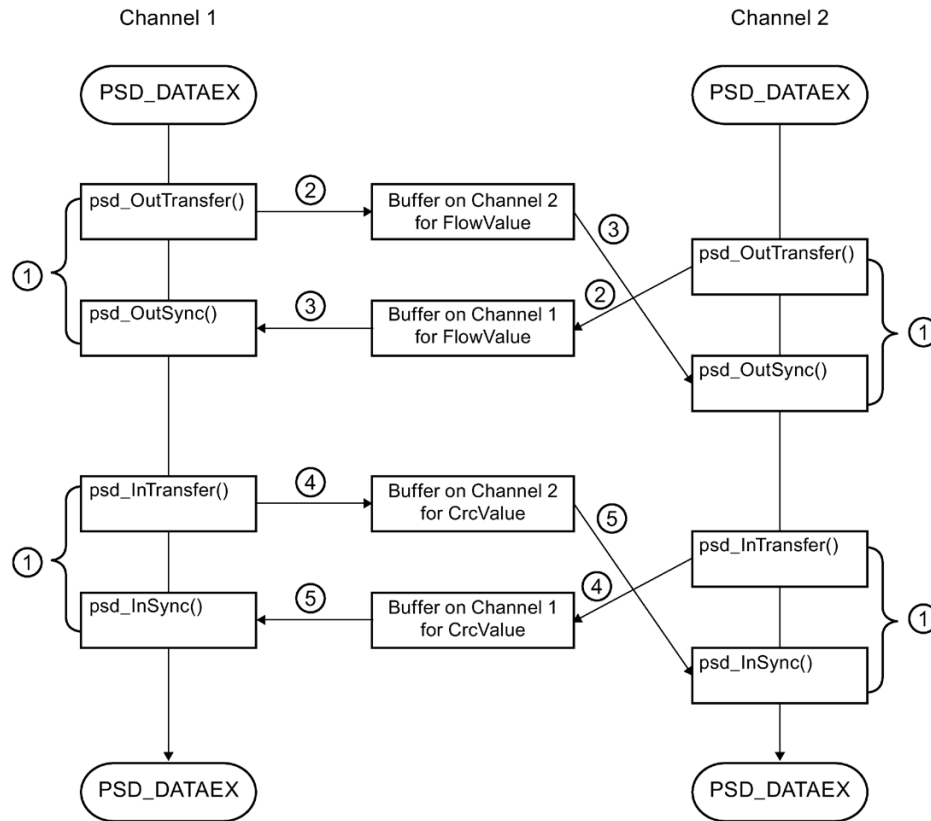
### 2.1.3 Tasks of the F-application:

Implementation of the F-application lies outside the PSD and has to fulfill the following tasks:

- Only the F-application controls the whole sequence. All PSD function calls provided in p\_api.h (see: Files of the PSD (Page 25)) can be used by the F-application. The F-application thus controls:
  - The initialization of the PSD
  - The parameterization with F-parameters
  - The configuration of the user data lengths
  - The processing of the F-output and F-input message frames
  - The reaction to start/stop events of the PROFIsafe communication
- The F-application distributes (with two channels) the data packets (configuration and parameter data, F-frames, etc.) to both channels with their synchronization capabilities.
- The F-application adopts the F-output data as well as specific bits from the PROFIsafe control byte from the PSD and processes them.
- The F-application generates the F-input data as well as specific bits for the PROFIsafe status byte and transfers them to the PSD. The F-input data are received by the PSD in the "Communication error" and "Timeout" error scenarios and sent to the F-host.

### 2.1.4 The synchronization interface

The following figure shows the sequence of synchronization in cyclic mode:



- ① The sequence in which the interface functions (Interface Description (Page 29)) of the F-application is monitored by the PSD.
- ② Call of the interface function `psd_OutTransfer(Instance, FlowValue)`
- ③ Call of the interface function `psd_OutSync(Instance, FlowValue)`
- ④ Call of the interface function `psd_InTransfer(Instance, CrcValue)`
- ⑤ Call of the interface function `psd_InSync(Instance, CrcValue)`

Figure 2-3 Process of synchronization used by PSD

In a first step the PSD uses the transfer functions to send the synchronization data to the respective other channel. You can find a description of the function in the section. Functions to be implemented at the 2-channel generation variant (Page 57).

In the second step, the PSD fetches data received in the meantime using the synchronization functions for the flow control or the CRC value. The synchronization function waits time monitored until the data arrive, and the PSD then compares them with the data of its own channel. This means that the data from the other channel must be available "in good time", i.e. within a device-specific synchronization timeout. Different data for flow control or a CRC value send the PSD to the `PSD_HARD_FAIL` state.

Ensure that the buffered data are read out only once respectively (for example by deleting from the buffer after the output).

### **! WARNING**

#### **Requirement: PSD\_RQ\_2\_01**

The F-application must provide a synchronization interface with time monitoring for 2-channel PSD generation. Mirroring (synchronization of a channel with itself) of the data to be exchanged is to be excluded.

## 2.2 States of the PSD

The state machine implemented in the PSD essentially operates with the following states:

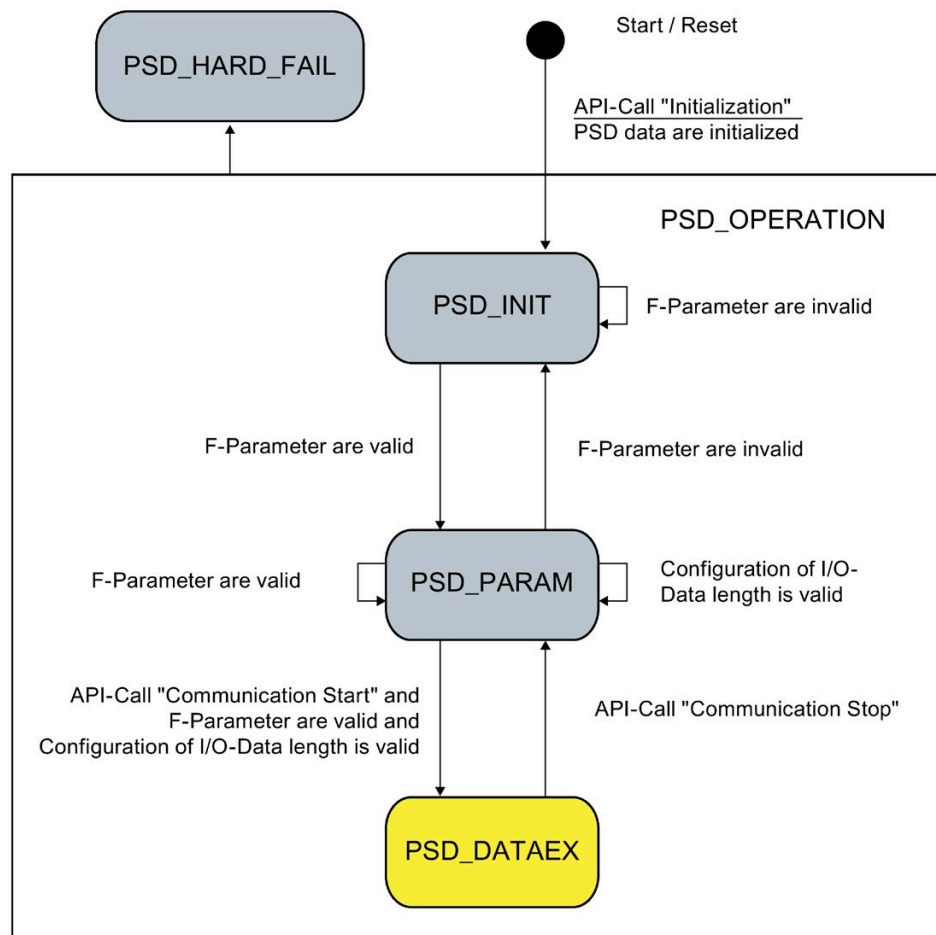


Figure 2-4 PSD states in a communication instance

### 2.2.1 Description of the states

#### PSD\_OPERATION

After successful initialization, the F-slave starts its operation, which is shown above as PSD\_OPERATION. This includes the following states relevant for the PROFIsafe communication.

#### PSD\_INIT

The PSD state is entered when the **psd\_InitInstance(...)** function has been executed for a communication instance without errors. The PSD instance is initialized correctly in this state, but it was not configured or called the **psd\_FParBuild(...)** function with invalid F-parameters.

#### PSD\_PARAM

The **psd\_FParBuild(...)** function was called with invalid F-parameters.

The configuration of the user data lengths must be made in this state with the PSD function **psd\_Config(...)**.

A PSD instance is ready for cyclic data exchange when:

- Valid F-parameters exist and
- A valid configuration of the user data lengths exists.

#### PSD\_DATAEX

The cyclic data exchange with the F-host has been started. The **psd\_Run(...)** function was called for the communication instance. The cyclic processing of input/output frames begins as soon as the toggle bit changes from 0 to 1 in the control byte of the output frame. New F-parameters or a reconfiguration of user data lengths are rejected in this PSD state.

#### PSD\_HARD\_FAIL

The PSD instance has established an inconsistency, probably due to a hardware fault. At a transition to the PSD state PSD\_HARD\_FAIL the PSD sets the interface variable "psd\_SafetyVar" to the value "P\_SAFETY\_CONST\_FALSE". The PSD instance calls the **hard\_err(...)** function of the F-application. With the **hard\_err(...)** function, the F-application sets the safe state (for example, the specific safety values in the output process data and the input frame section) for all communication instances.

## 2.2.2 Description of the state transitions

### START → PSD\_INIT

After PowerOn, the F-application initializes the respective communication instance by calling **psd\_InitInstance(...)**. Once it runs error-free, the communication instance is in the PSD\_INIT state.

### F-CYCLE → PSD\_HARD\_FAIL

Any inconsistency or any hardware fault detected by the PSD, always results in the PSD\_HARD\_FAIL state for all PSD instances.

### PSD\_INIT → PSD\_PARAM

The stack signals the receipt of F-parameters to the F-application. The F-application then configures the communication instance by calling the **psd\_FParBuild(...)** function. The PSD\_PARAM state is reached when valid F-parameters exist.

### PSD\_PARAM → PSD\_INIT

The stack signals the receipt of F-parameters to the F-application. The F-application then reconfigures the communication instance by calling the **psd\_FParBuild(...)** function. The **psd\_FParBuild(...)** function has detected an invalid F-parameter.

### PSD\_PARAM → PSD\_PARAM

After receipt of new F-parameters from the stack, the F-application reconfigures the communication instance. The PSD\_PARAM state remains as long as valid F-parameters are present. (See section F-input or F-output user data area (Page 77))

### PSD\_PARAM → PSD\_PARAM

The lengths for the input and output user data must be set with the **psd\_Config(...)** function for each communication instance. The maximum permissible lengths are set for each instance in p\_c\_config.h. The optional values passed by the F-application must be less than or equal to the values of p\_c\_config.h.

### PSD\_PARAM → PSD\_DATAEX

The stack signaled to the F-application the transition to cyclic data exchange for a particular communication instance with **[RUN]**. The F-application then calls the **psd\_Run(...)** function.

### PSD\_DATAEX → PSD\_PARAM

The stack signaled to the F-application the exit from cyclic data exchange for a particular communication instance with **[STOP]**. The F-application then calls the **psd\_Stop(...)** function.

### 2.2.3 Activities of the PSD in the PSD\_DATAEX state

#### 2.2.3.1 Processing of the output message frames

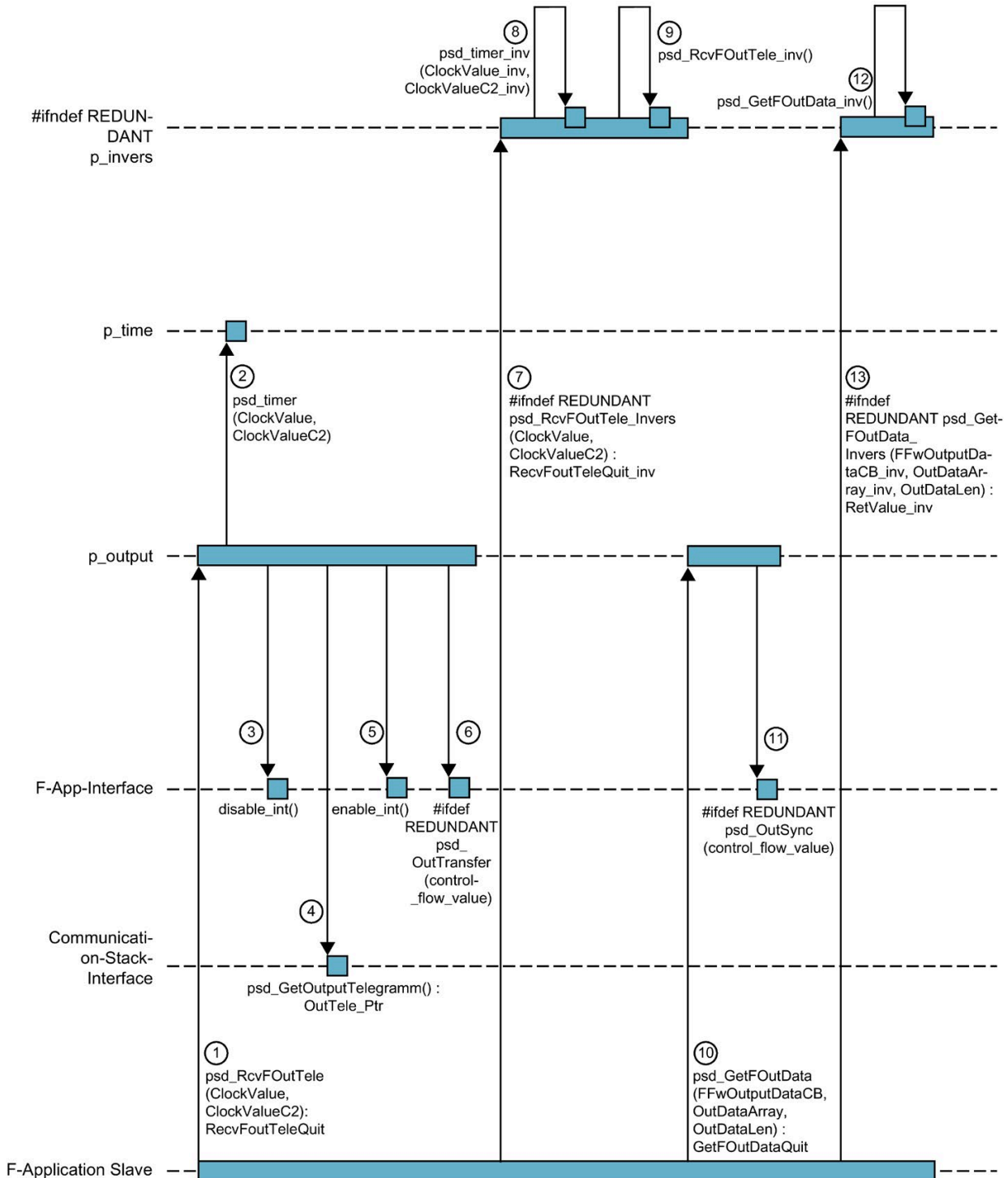


Figure 2-5 Processing the cyclic output frame

- ① The F-application calls the **psd\_RecvFOutTele(...)** PSD function in order to receive and check an output frame from the stack interface. It passes its independently generated, current time values to the PSD.
- ② The PSD uses the the passed time values to calculate whether or not there is a timeout. This means the F-cycle, that is the F-host < > F-device communication time, is monitored. Note that the module cycle and the performance of the bus communication (e.g. PROFINET) impact the F-monitoring time.
- ③ The PSD request an interrupt disable for the consistent transfer of the output frame.
- ④ The PSD request a pointer to the buffer with the output frame from the stack interface. The PSD adopts the output message frame into its internal data structure. In the case of 1 channel the PSD also fills the inverse data structure with the output message frame.
- ⑤ Revoking the interrupt disable.  
The PSD checks the output frame in accordance with the PROFIsafe profile.
- ⑥ In a 2 channel scenario, the PSD sends its current program flow control value to the other channel.
- ⑦ In a 1 channel scenario, the inverse part of the F-application calls the **psd\_RecvFOutTele\_Invers(...)** PSD function to check the inverted output frame. The inverse part of the F-application passes its independently generated, inverted values to the PSD. This means the F-cycle, that is the F-host < > F-device communication time, is monitored. Note that the module cycle and the performance of the bus communication (e.g. PROFINET) impact the F-monitoring time.
- ⑧ The PSD uses the the passed time values to inversely calculate whether or not there is a timeout.
- ⑨ The PSD checks the inverted output frame.
- ⑩ The F-application calls the **psd\_GetFOutData(...)** function to obtain the output data and the bit information from the PROFIsafe control .
- ⑪ In the 2-channel case, the PSD reads the program flow control value of the other channel from the F-application.  
In the the 2-channel case, the PSD compares its program flow control value with that of the other channel. Then it fills the buffer of the F-application with the output data and bit information from the PROFIsafe control byte.  
The F-Application checks the return value of **psd\_GetFOutData(...)** and processes the output data..
- ⑫ In a 1 channel scenario, the inverse part of the F-application calls the **psd\_GetFOutData\_Invers(...)** function to obtain the inverted output data and bit information from the PROFIsafe control byte.
- ⑬ The PSD fills the buffer of the F-application with the output data and bit information from the PROFIsafe control byte.

**WARNING****Requirement: PSD\_RQ\_0\_07**

Impermissible return values have to induce the **HARD\_FAIL** state of the F-application.

 **WARNING**

**Requirement: PSD\_RQ\_0\_18**

The F-application has to take suitable measures in order to recognize different F-output user data or control byte identifiers between the two channels (for example due to an error in the output storage area). This can, for example, occur through comparison of the data transferred by the PSD. The test has to be carried before output to the process.

 **WARNING**

**Requirement: PSD\_RQ\_0\_19**

If the PSD signals a communication error, the received control byte is invalid and the control byte identifiers transferred to the F-application CB\_B0\_iParEN, CB\_B1\_OAReq and CB\_B6\_ChF\_Ack are not to be evaluated.

## 2.2.3.2 Processing of the input message frames

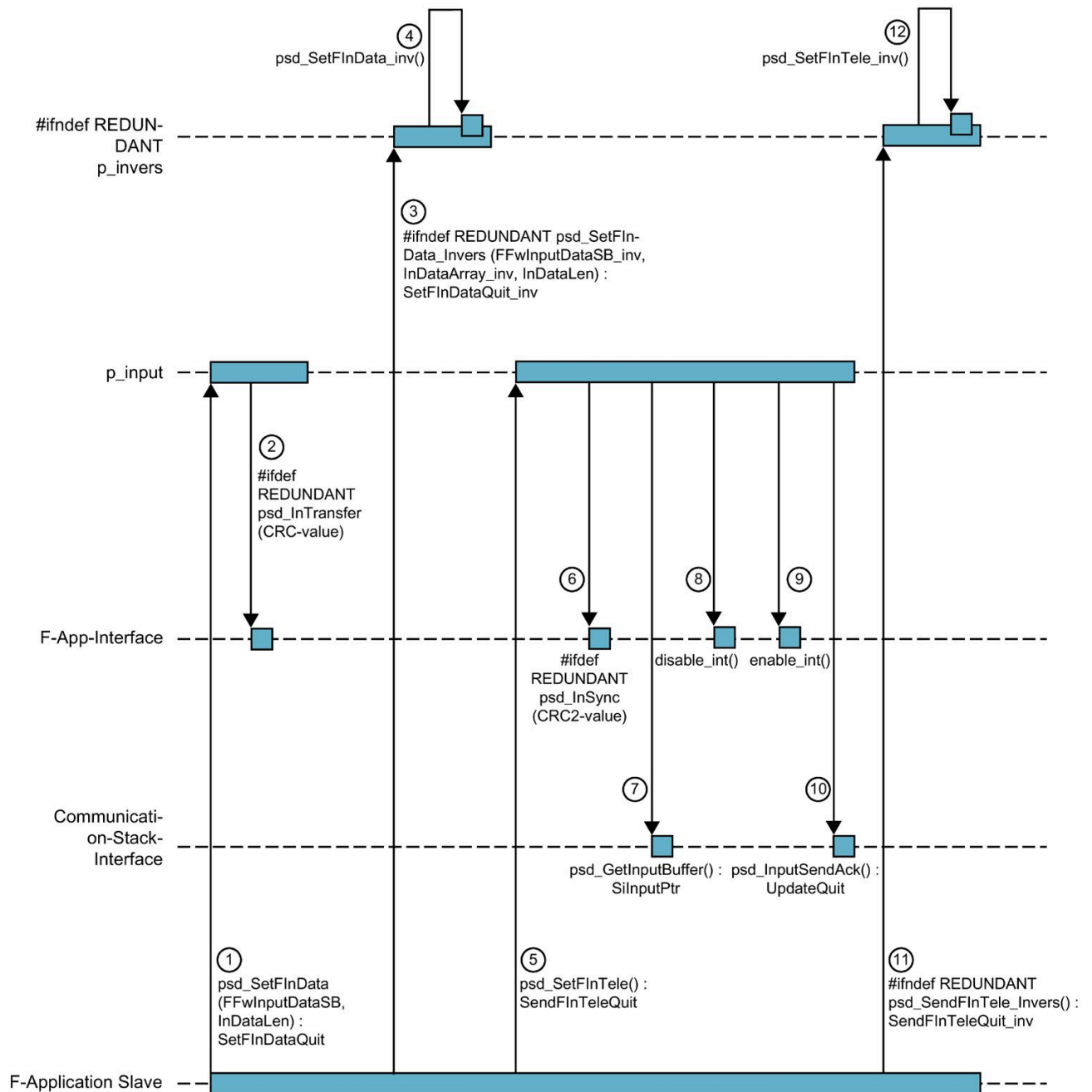


Figure 2-6 Processing the cyclical input frames

- ① The F-application calls the **psd\_SetFlInData(...)** function to transfer the input data. The PSD takes the input data and specific bit information for the structure of the PROFIsafe status byte from the F-application and forms the input frame.
- ② In a 2 channel scenario, the PSD sends the input frame CRC to the other channel with **psd\_InTransfer(...)**.

## 2.2 States of the PSD

- ③ In the case of 1 channel the "inverse part" of the F-application calls the **psd\_SetFInData\_Invers(...)** function.
- ④ The PSD takes the inverted input data and bit information from the F-application and forms the inverted input frame.
- ⑤ The F-application calls the **psd\_SendFInTele(...)** function to send the input frame.
- ⑥ The PSD checks the input message frame in accordance with the PROFIsafe Profile. It extends the input frame with the CRC of the other channel, which it reads in the 2-channel case with the **psd\_InSync(...)** function. In the case of 1 channel the PSD uses the CRC from its inverse processing.
- ⑦ The PSD uses the **psd\_GetInputBuffer(...)** function to request a send buffer for the input frame from the stack interface.
- ⑧ The PSD requests an interrupt disable from the F-application for consistent filling of the send buffer.
- ⑨ Release of the interrupt disable.
- ⑩ The PSD uses the **psd\_InputSendAck(...)** function to signal to the stack interface that the send buffer is filled and thus ready for sending to the F-host.
- ⑪ In a 1 channel scenario, the "inverse part" of the F-application calls the **psd\_SendFInTele\_Invers(...)** function. It compares the program flow control values of the normal and inverted calculation at the end.
- ⑫ The PSD generates the inverted program flow control value in **psd\_SendFInTele\_inv(...)**.



### WARNING

#### Requirement: PSD\_RQ\_0\_07

Impermissible return values have to induce the **HARD\_FAIL** state of the F-application.

## Files of the PSD

The following tables list all the source files that can be found on the CD ROM for the PSD. The files contain, among other things, the interface functions and bit information on the structure of the status byte, which are briefly described below. You can find a detailed interface description in the sections below.

---

### Note

Call the input and output functions of the PROFIsafe driver cyclically.

- psd\_RecvFOutTele(...)
- psd\_GetFOutData(...)
- psd\_SetFInData(...)
- psd\_SendFInTele(...)

The above processing order must be adhered to.

---

### 3.1 Files that cannot be changed

You may not modify the following files (file names and file contents):

| File       | Interface function        | Task                                                                               |
|------------|---------------------------|------------------------------------------------------------------------------------|
| p_crc.c    | psd_CRC16(...)            | Calculate a 16-bit CRC across a storage area                                       |
|            | psd_CRC24(...)            | Calculate a 24-bit CRC across a storage area                                       |
|            | psd_CRC32(...)            | Calculate a 32-bit CRC across a storage area                                       |
| p_init.c   | psd_InitInstance(...)     | Initialize a PSD instance                                                          |
|            | psd_SetFID(...)           | Activate a fault-insertion test                                                    |
| p_state.c  | psd_GetState(...)         | Output a PSD state                                                                 |
|            | psd_GetPSD_Version(...)   | Output a PSD version                                                               |
|            | psd_Stop(...)             | Terminate the cyclic data exchange of the PSD                                      |
|            | psd_Run(...)              | Set the PSD to the state "PSD_DATAEX"                                              |
|            | psd_Falsify_InTelCRC(...) | Falsifies the CRC value of the F-input frame for communication Instance            |
| p_input.c  | psd_SetFInData(...)       | Transfer F-input data to the PSD                                                   |
|            | psd_SendFInTele(...)      | Set up F-input message frame and transfer to the stack interface                   |
| p_output.c | psd_RecvFOutTele(...)     | Receive F-output message frame from stack interface and check                      |
|            | psd_GetFOutData(...)      | Pass F-output data and the result of the output telegram test of the F-application |
|            | psd_GetTraceData(...)     | Pass diagnostics data of the F-application                                         |
| p_param.c  | psd_FParBuild(...)        | Receive F-parameters from stack interface and check                                |

## 3.1 Files that cannot be changed

| File       | Interface function | Task                                                                                                                                                                                                                |
|------------|--------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|            | psd_GetFPar(...)   | Pass F_SIL, F_iPar_CRC and the type of passivation (channel/module granular) of the F-application                                                                                                                   |
|            | psd_Config(...)    | Configure user data length                                                                                                                                                                                          |
|            | psd_GetConfig(...) | Pass configured length of input and output user data as well as the PROFIsafe trailer length of the F-application                                                                                                   |
| p_time.c   | ---                | Internal timer management                                                                                                                                                                                           |
| p_64bit.c  | ---                | Emulation of the 64-bit arithmetic. Used to calculate the monitoring number for the bit F_Prm_Flag1.F_CRC_Seed = 1 in the F-parameters, if the used C compiler does not support the data type "unsigned long long". |
| p_types.h  | ---                | Declaration of the PSD base data types                                                                                                                                                                              |
| p_api.h    | ---                | Declaration of the PSD interface functions                                                                                                                                                                          |
| p_global.h | ---                | Global type declarations for the F-application                                                                                                                                                                      |
| p_local.h  | ---                | PSD-local type declarations                                                                                                                                                                                         |

Additionally for 1-channel generation variant:

| File       | Interface function           | Task                                                                               |
|------------|------------------------------|------------------------------------------------------------------------------------|
| p_invers.c | psd_GetFOutData_Invers(...)  | Return value of the inverse calculation path of the psd_GetFOutData(...) function  |
|            | psd_RecvFOutTele_Invers(...) | Return value of the inverse calculation path of the psd_RecvFOutTele(...) function |
|            | psd_SendFInTele_Invers(...)  | Return value of the inverse calculation path of the psd_SendFInTele(...) function  |
|            | psd_SetFInData_Invers(...)   | Return value of the inverse calculation path of the psd_SetFInData(...) function   |
|            | psd_InitInstance_Invers(...) | Return value of the inverse calculation path of the psd_InitInstance(...) function |
|            | psd_FParBuild_Invers(...)    | Return value of the inverse calculation path of the psd_FParBuild(...) function    |
|            | psd_Config_Invers(...)       | Return value of the inverse calculation path of the psd_Config(...) function       |
|            | psd_GetState_Invers(...)     | Return value of the inverse calculation path of the psd_GetState(...) function     |
|            | psd_Stop_Invers(...)         | Return value of the inverse calculation path of the psd_Stop(...) function         |
|            | psd_Run_Invers(...)          | Return value of the inverse calculation path of the psd_Run(...) function          |
|            | psd_GetFPar_Invers(...)      | Return value of the inverse calculation path of the psd_GetFPar(...) function      |
|            | psd_GetConfig_Invers(...)    | Return value of the inverse calculation path of the psd_GetConfig(...) function    |

**WARNING**

**Requirement: PSD\_RQ\_0\_02**

You may not change PSD files that are protected with a CRC value.

The **crc.exe** tool is available to verify that the original PSD sources have been used. The tool calculates a signature for each PSD file as well as a collective signature for all PSD sources.

Read the provided "readme.pdf" file /7/ about the installation and use of **crc.exe**.

Table 3- 1 The CRC values of the PSD sources for a 1-channel HW structure

| No. | File       | CRC       |
|-----|------------|-----------|
| 1   | p_64bit.c  | 3FB9 694B |
| 2   | p_crc.c    | A09D A7A4 |
| 3   | p_init.c   | 1A2C DB5C |
| 4   | p_input.c  | 6689 A9A4 |
| 5   | p_invers.c | 10A0 49C3 |
| 6   | p_output.c | C677 C3AE |
| 7   | p_param.c  | F18B A903 |
| 8   | p_state.c  | EC01 FA34 |
| 9   | p_time.c   | 47AA C1EF |
| 10  | p_api.h    | 2671 D781 |
| 11  | p_global.h | ABF1 CE79 |
| 12  | p_local.h  | 36FE 4D72 |
| 13  | p_types.h  | 538F 49A9 |
| 14  | p_fit.h    | 8F67 F40C |
|     |            |           |
| 15  | Total:     | D89E 0019 |

Table 3- 2 The CRC values of the PSD sources for a 2-channel HW structure

| No. | File       | CRC       |
|-----|------------|-----------|
| 1   | p_64bit.c  | 3FB9 694B |
| 2   | p_crc.c    | A09D A7A4 |
| 3   | p_init.c   | 1A2C DB5C |
| 4   | p_input.c  | 6689 A9A4 |
| 5   | p_output.c | C677 C3AE |
| 6   | p_param.c  | F18B A903 |
| 7   | p_state.c  | EC01 FA34 |
| 8   | p_time.c   | 47AA C1EF |
| 9   | p_api.h    | 2671 D781 |
| 10  | p_global.h | ABF1 CE79 |
| 11  | p_local.h  | 36FE 4D72 |
| 12  | p_types.h  | 538F 49A9 |
| 13  | p_fit.h    | 8F67 F40C |
|     |            |           |
| 14  | Total:     | 5004 4017 |

**WARNING****Requirement: PSD\_RQ\_0\_03**

The CRC values of the PSD files (individual and overall CRC) have to agree with the values of the CRC check tool.

## 3.2 Files that can be changed

You need to adapt these files for the integration of the PSD in your F-application:

| File               | Task                                                                                                             |
|--------------------|------------------------------------------------------------------------------------------------------------------|
| p_c_config.h       | Definition of PSD generation parameters, maximum lengths of the F-input and F-output user data memory attributes |
| p_c_crc_m.h        | Memory allocation of the CRC table                                                                               |
| p_c_crc_b.h        | Restoring of the default memory model                                                                            |
| p_c_fapplication.c | Example code for an F-application that uses the PSD                                                              |
| p_c_fapplication.h | Function prototypes of the F-application interface and sync interface                                            |
| p_c_si.c           | Example implementation of functions of the stack interface that invokes the PSD                                  |
| p_c_si.h           | Function prototypes of the stack interface                                                                       |

## 3.3 Commissioning procedure

The following steps are to be carried out in order to use the PSD:

- Implement surrounding FW modules (stack interface synchronization interface, F-application) so that all functions required by the PSD can be called.
- The p\_api.h file is included in the F-application
- The generation parameters and maximum user data lengths are set in the p\_c\_config.h file.
- The desired memory model and the memory classes are set in the p\_c\_config.h file, see sections Memory attributes (Page 78) and Memory model for the CRC tables (Page 78).

**Note**

A pseudocode /3/ is provided to illustrate the PSD integration (files: **p\_c\_pseudo.c**, **p\_c\_fapplication.c**, **p\_c\_si.c**).

## Interface Description

### 4.1 PSD interface functions

The PSD can be used in different system environments. This is achieved with the following features:

- You can adapt a specific F-application or stack interface by programming the interface functions from `p_c_fapplication.h` or `p_c_si.h`.  
This procedure does not change the certified source code of the PSD.
- The PSD detects the byte order "big endian" or "little endian" of hardware it uses. The PSD is not tested for 2-channel F-slaves, in which the two channels are operated with a different byte order. This must be taken into consideration for the synchronization of the F-application.
- With the memory attributes for the memory segment control that be set in `p_c_config.h`, the storage location of data and code can be configured to match the tool chain.
- Additional settings for hardware architecture or memory model can be made with the generation parameters in `p_c_config.h`.

#### WARNING

##### Requirement: PSD\_RQ\_0\_04

You may use only those PSD interface functions that are declared in the `p_api.h` file.

#### WARNING

##### Requirement: PSD\_RQ\_0\_05

You may only use the PSD interface structures that are declared in the `p_global.h` file.

#### WARNING

##### Requirement: PSD\_RQ\_0\_06

You have to use the Defines from `p_global.h` for the evaluation of the PSD returns

#### WARNING

##### Requirement: PSD\_RQ\_0\_07

Impermissible return values have to induce the **HARD\_FAIL** state of the F-application.

### 4.1.1 psd\_InitInstance(...)

Initialize a PSD instance.

The "DynamicTeleLenConf" parameter is used to enable or disable dynamic configuration of the user data lengths in the PSD\_PARAM state, which is performed with **psd\_Config(...)**. If disabled, the maximum user data lengths defined in p\_c\_config.h are used.

|                   |                                                                            |
|-------------------|----------------------------------------------------------------------------|
| <b>Function:</b>  | <b>psd_InitInstance(...)</b>                                               |
| <b>Prototype:</b> | P_Byte psd_InitInstance<br>(P_Word Inst_ID,<br>P_Byte DynamicTeleLenConf); |

| <b>Transfer parameter:</b> | <b>Value range</b>                     | <b>Meaning</b>                                                                                                                        |
|----------------------------|----------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| Inst_ID                    | INST_01...INST_X<br>X <= NOF_INSTANCES | Instance ID                                                                                                                           |
| DynamicTeleLenConf         | DYNAMIC_TELE_LEN_CONF_ON               | The maximum values must be set in the "p_c_config.h" file. Current data lengths of the instance are set with <b>psd_Config(...)</b> . |
|                            | DYNAMIC_TELE_LEN_CONF_OFF              | Data lengths of the instance are specified in "p_c_config.h"                                                                          |

| <b>Return parameter:</b> | <b>Value range</b>   | <b>Meaning</b>                                                                                      |
|--------------------------|----------------------|-----------------------------------------------------------------------------------------------------|
| P_Byte                   | INIT_INSTANCE_OK     | Initialization successful                                                                           |
|                          | INIT_INSTANCE_NOT_OK | Length of the data types used by the PSD deviates from the expected length.<br>Hard error situation |

#### **WARNING**

##### **Permissible return values**

From the point of view of the F-application, only the following return values are permitted:

- INIT\_INSTANCE\_OK

##### **Requirement: PSD\_RQ\_0\_07**

Impermissible return values have to induce the **HARD\_FAIL** state of the F-application.

### 4.1.2 psd\_FParBuild(...)

Parameterization of a PSD instance.

The F-parameters are read from the stack interface and via **psd\_GetFParPtr(...)** in the function and then a plausibility check is performed.

The function parameters (source, destination address, SIL) passed in a structure (Fw\_FPar), which must be determined independent of the F-parameters by the F-application, are compared with the read F-parameters by the PSD.

The structuring of Fw\_FPar depends on the "COMPARE\_F\_SOURCE\_ADD" setting in p\_c\_config.h. The structure additionally contains the F\_Source\_Address with the Define.

The function can be called in the (instance-specific) PSD states PSD\_INIT and PSD\_PARAM.

In the case of invalid F-parameters, the PSD state PSD\_INIT is assumed. In the case of valid F-parameters, the PSD state PSD\_PARAM is assumed.

In the state PSD\_DATAEX parameterization with F-parameters is rejected.

|                  |                                                                                 |
|------------------|---------------------------------------------------------------------------------|
| <b>Function:</b> | <b>psd_FParBuild(...)</b>                                                       |
| Prototype:       | P_Byte psd_FParBuild<br>(P_Word Inst_ID,<br>Fw_FPar FwPar,<br>P_Word psd_CRC3); |

| Transfer parameter:                              | Value range                                          | Meaning                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
|--------------------------------------------------|------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Inst_ID                                          | INST_01...INST_X<br>X <= NOF_INSTANCES               | Instance ID                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| FwPar.FSourceAdd<br>FwPar.FDestAdd<br>FwPar.FSil | See "Layout of the device parameter structure" below |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| psd_CRC3                                         | 0 - 0xFFFF                                           | 16-bit iParameter CRC<br>If "F_Check_iPar" = 1 in the F-parameters, the CRC iParameter is included in the calculation of the F-Parameter CRC. In this case, psd_CRC3 may not be "0".<br>If "F_Check_iPar" = 0 or "F_Block_ID" = 001 or 011, (F_iPar_CRC is contained in the F-parameters), psd_CRC3 not evaluated.<br>If IPar CRC = 0, the state machine changes to process data exchange with failsafe values. This special state (FSCP test mode) is described in more detail in the PSD standard section <b>8.1.7 F_iPar_CRC (value of iPar_CRC across iParameters)</b> . |

| Return parameter: | Value range                 | Meaning                                                                                         |
|-------------------|-----------------------------|-------------------------------------------------------------------------------------------------|
| P_Byte            | FPAR_BUILD_OK               | Parameterization successful                                                                     |
|                   | FPAR_F_DEST_ADD_MISMATCH    | The configured destination address does not match the destination address of the F-application. |
|                   | FPAR_F_DEST_ADD_NOT_VALID   | Invalid target address 0 or 0xFFFF)                                                             |
|                   | FPAR_F_SOURCE_ADD_MISMATCH  | The configured source address does not match the source address of the F-application.           |
|                   | FPAR_F_SOURCE_ADD_NOT_VALID | Invalid source address 0 or 0xFFFF)                                                             |
|                   | FPAR_F_WD_TIME_NULL         | Watchdog time is invalid (= 0)                                                                  |
|                   | FPAR_F_SIL_ERR              | Configured SIL class is higher than the SIL class that the F-application has passed to the PSD  |
|                   | FPAR_CRC_LENGTH             | Maximum length of the user data from p_c_config.h does not match the parameterized CRC width    |
|                   | FPAR_VERSION_ERR            | Invalid F-parameter version                                                                     |
|                   | FPAR_BLOCK_ID_ERR           | Invalid block ID                                                                                |
|                   | FPAR_CRC1_ERR               | F-parameter CRC – error                                                                         |
|                   | FPAR_BUILD_NOT_POSSIBLE     | Parameterization (in RUN) not possible                                                          |
|                   | FPAR_BUILD_NOT_OK           | Parameterization inconsistent                                                                   |

Structure of the device parameters when Define "COMPARE\_F\_SOURCE\_ADD" is set in p\_c\_config.h:

| Code                                                                                                                                                                                                                                                                                                                                                          | Meaning                                                                                                                                                                                                                                                                                                                               |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> #ifdef REDUNDANT typedef struct Fw_FPar_t {     P_Word      FSourceAdd     P_Word      FDestAdd     P_Byte      FSil } Fw_FPar; #else typedef struct Fw_FPar_t {     P_Word      FSourceAdd     P_Word      FSourceAdd_inv     P_Word      FDestAdd     P_Word      FDestAdd_inv     P_Byte      FSil     P_Byte      FSil_inv } Fw_FPar; #endif </pre> | <ul style="list-style-type: none"> <li>• FSourceAdd: F-host address</li> <li>• FSourceAdd_inv: F-host address inverted</li> <li>• FDestAdd: F-device address</li> <li>• FDestAdd_inv: Inverted F-device address</li> <li>• FSil: max. SIL of the F-application</li> <li>• FSil_inv: Inverted max. SIL of the F-application</li> </ul> |

 **WARNING**

**Permissible return values**

From the point of view of the F-application, only the following return values are permitted:

- FPAR\_BUILD\_OK
- FPAR\_F\_DEST\_ADD\_MISMATCH
- FPAR\_F\_DEST\_ADD\_NOT\_VALID
- FPAR\_F\_SOURCE\_ADD\_MISMATCH
- FPAR\_F\_SOURCE\_ADD\_NOT\_VALID
- FPAR\_F\_WD\_TIME\_NULL
- FPAR\_F\_SIL\_ERR
- FPAR\_CRC\_LENGTH
- FPAR\_VERSION\_ERR
- FPAR\_BLOCK\_ID\_ERR
- FPAR\_CRC1\_ERR
- FPAR\_BUILD\_NOT\_POSSIBLE
- FPAR\_BUILD\_NOT\_OK

**Requirement: PSD\_RQ\_0\_07**

Impermissible return values have to induce the **HARD\_FAIL** state of the F-application.

 **WARNING**

**Requirement: PSD\_RQ\_0\_09**

The SIL of the F-parameters may not be greater than the SIL of the F-slave. The SIL of the F-slave must be determined from the F-parameters independent of the SIL.

 **WARNING**

**Requirement: PSD\_RQ\_0\_10**

The F-address has to be stored in the F-application. Extraction of the F-address or other values from the F-parameters is not permitted.

### 4.1.3 psd\_Config(...)

Configuration of the I/O data lengths for input and output user data for a PSD instance.

The I/O data lengths for the input and output user data can be configured as static or dynamic according to the setting in **psd\_InitInstance(...)**.

#### Variant "static"

The **psd\_Config(...)** function uses the lengths defined in p\_c\_config.h (OUTDATA\_MAX\_LEN\_INST\_xx / INDATA\_MAX\_LEN\_INST\_yy). It is not possible to reconfigure to a smaller length.

#### Variant "dynamic":

The **psd\_Config(...)** function uses the lengths from p\_c\_config.h as maximum values. Reconfiguring to smaller lengths is possible in PSD\_PARAM state.

When the I/O data lengths are configured, the lengths are read out via the **psd\_GetConfigPtr(...)** function.

When a data length exceeds the maximum value set in p\_c\_config.h, the PSD goes to the instance-global PSD\_HARD\_FAIL state.

The **psd\_Config(...)** function must be called in the PSD\_PARAM PSD state. Successful implementation is a prerequisite for the transition to the PSD\_DATAEX state, even if the "static" variant was set. **psd\_FParBuild(...)** must have been executed successfully prior to the call.

Configuration of the I/O data lengths is rejected in the PSD\_DATAEX and PSD\_INIT states with FPAR\_CONFIG\_NOT\_POSSIBLE.

|            |                                     |
|------------|-------------------------------------|
| Function:  | <b>psd_Config(...)</b>              |
| Prototype: | P_Byte psd_Config (P_Word Inst_ID); |

| Transfer parameter: | Value range                           | Meaning     |
|---------------------|---------------------------------------|-------------|
| Inst_ID             | INST_01...INST_X<br>X ≤ NOF_INSTANCES | Instance ID |

| Return parameter: | Value range              | Meaning                                                          |
|-------------------|--------------------------|------------------------------------------------------------------|
| P_Byte            | FPAR_CONFIG_OK           | Configuration successful                                         |
|                   | FPAR_CONFIG_NOT_POSSIBLE | Configuration not possible in the PSD_DATAEX and PSD_INIT states |
|                   | FPAR_CONFIG_NOT_OK       | Hard error situation                                             |

|                                                                                                                                                                                                                                                                                                                                                                                              |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  <b>WARNING</b>                                                                                                                                                                                                                                                                                             |
| <p><b>Permissible return values</b></p> <p>From the point of view of the F-application, only the following return values are permitted:</p> <ul style="list-style-type: none"> <li>• FPAR_CONFIG_OK</li> <li>• FPAR_CONFIG_NOT_POSSIBLE</li> </ul> <p><b>Requirement: PSD_RQ_0_07</b></p> <p>Impermissible return values have to induce the <b>HARD_FAIL</b> state of the F-application.</p> |

The table below shows the structuring of the I/O data lengths, which are read with psd\_GetConfigPtr(...) (Page 54):

| P_Byte IORange[2] | Meaning                                                                                                                                                                                                     |
|-------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| IORange[0]        | <p>Number of bytes of the input message frame</p> <p>(At a 3-byte CRC: 4 bytes PROFIsafe trailer + 0..12 user data bytes)</p> <p>(At a 4-byte CRC: 5 bytes PROFIsafe trailer + 0..123 user data bytes)</p>  |
| IORange[1]        | <p>Number of bytes of the output message frame</p> <p>(At a 3-byte CRC: 4 bytes PROFIsafe trailer + 0..12 user data bytes)</p> <p>(At a 4-byte CRC: 5 bytes PROFIsafe trailer + 0..123 user data bytes)</p> |

#### 4.1.4 psd\_Run(...)

Enable cyclic data exchange and change of an instance in the "PSD\_DATAEX" state.

Condition for changing to the PSD\_DATAEX state:

**psd\_FParBuild(...)** and **psd\_Config(...)** have been successfully executed.

Cyclic data exchange starts as soon as the toggle bit in the PROFIsafe control byte changes from 0 to 1. This test is performed in the context of **psd\_RecvFOutTele(...)**.

The F-input data from the F-application are received by the PSD in the F-input frame as soon as the cyclic data exchange is started.

|                   |                                  |
|-------------------|----------------------------------|
| <b>Function:</b>  | <b>psd_Run(...)</b>              |
| <b>Prototype:</b> | P_Byte psd_Run (P_Word Inst_ID); |

| <b>Transfer parameter:</b> | <b>Value range</b>                     | <b>Meaning</b> |
|----------------------------|----------------------------------------|----------------|
| Inst_ID                    | INST_01...INST_X<br>X <= NOF_INSTANCES | Instance ID    |

| <b>Return parameter:</b> | <b>Value range</b> | <b>Meaning</b>                                                                                     |
|--------------------------|--------------------|----------------------------------------------------------------------------------------------------|
| P_Byte                   | PSD_DATAEX         | The cyclic data exchange can be started by calling <b>psd_RecvFOutTele(...)</b> .                  |
|                          | PSD_PARAM          | Cyclic data exchange cannot be started. Configuration of the user data lengths missing.            |
|                          | PSD_INIT           | No F-parameter assignment                                                                          |
|                          | PSD_HARD_FAIL      | Inconsistent instance ID, only in 1-channel HW structure: Inconsistent program flow control value. |

#### WARNING

##### Permissible return values

From the point of view of the F-application, only the following return values are permitted:

- PSD\_DATAEX
- PSD\_PARAM
- PSD\_INIT

**Requirement: PSD\_RQ\_0\_07**

Impermissible return values have to induce the **HARD\_FAIL** state of the F-application.

### 4.1.5 psd\_Stop(...)

Ending the cyclic data exchange of an instance. If the function is called in the "PSD\_DATAEX" state, it sets the PSD to the "PSD\_PARAM" state. The F-output user data are set by the PSD to 0. The F-input user data are not set to 0 by the PSD.

If the call is effected from a different state, the current state is retained.

|                   |                                   |
|-------------------|-----------------------------------|
| <b>Function:</b>  | <b>psd_Stop(...)</b>              |
| <b>Prototype:</b> | P_Byte psd_Stop (P_Word Inst_ID); |

| <b>Transfer parameter:</b> | <b>Value range</b>                     | <b>Meaning</b> |
|----------------------------|----------------------------------------|----------------|
| Inst_ID                    | INST_01...INST_X<br>X <= NOF_INSTANCES | Instance ID    |

| <b>Return parameter:</b> | <b>Value range</b> | <b>Meaning</b>                                                                                     |
|--------------------------|--------------------|----------------------------------------------------------------------------------------------------|
| P_Byte                   | PSD_PARAM          | Call from PSD_DATAEX: The cyclic data exchange is stopped. PSD is parametered and configured       |
|                          | PSD_HARD_FAIL      | Inconsistent instance ID, only in 1-channel HW structure: Inconsistent program flow control value. |

#### **WARNING**

##### **Permissible return values**

From the point of view of the F-application, only the following return values are permitted:

- PSD\_PARAM

##### **Requirement: PSD\_RQ\_0\_07**

Impermissible return values have to induce the **HARD\_FAIL** state of the F-application.

### 4.1.6 psd\_RecvFOutTele(...):

---

**Note**

Call this function cyclically in your F-application.

---

Reception of F-output frame from the F-host in the PSD for the specified instance.

The PSD calculates whether or not there is a timeout with the passed 1-ms time values of the two channels.

The function takes the cyclic F-output frame in the PSD OutData buffer.

In the 2-channel generation variant, the ClockValue time value is generated by the timer on CPU1 and the ClockValueC2 time value is generated by the timer on CPU2.

The two time values (ClockValue, ClockValueC2) must also be generated in the 1-channel generation variant with two independent hardware timers.

The tolerable difference between the two values must be determined and verified by the F-application.

The PSD uses the **psd\_GetOutputTelegram(...)** function to read the F-output frame and checks that the frame conforms to the rules of the PROFIsafe profile 1/.

The acquisition of the output frame is bracketed with the **disable\_int(...)** and **enable\_int(...)** functions in order to ensure consistency.

|                  |                                                                                         |
|------------------|-----------------------------------------------------------------------------------------|
| <b>Function:</b> | <b>psd_RecvFOutTele(...)</b>                                                            |
| Prototype:       | P_Byte psd_RecvFOutTele<br>(P_Word Inst_ID,<br>P_Word ClockValue, P_Word ClockValueC2); |

| Transfer parameter: | Value range                            | Meaning          |
|---------------------|----------------------------------------|------------------|
| Inst_ID             | INST_01...INST_X<br>X <= NOF_INSTANCES | Instance ID      |
| ClockValue          | 0...0xFFFF                             | 1 ms timer value |
| ClockValueC2        | 0...0xFFFF                             | 1 ms timer value |

| Return parameter: | Value range           | Meaning                                      |
|-------------------|-----------------------|----------------------------------------------|
| P_Byte            | RECV_FOUT_TELE_OK     | Output message frame applied                 |
|                   | RECV_FOUT_TELE_NOT_OK | Inconsistency detected, Hard Error situation |

|                                                                                                                                                                                                                                                                                                                                                   |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  <b>WARNING</b>                                                                                                                                                                                                                                                  |
| <b>Time values</b><br>Independently generated time values.<br><b>Requirement: PSD_RQ_0_16</b><br>The two timer values passed by the F-application to the PSD have to be generated independently of each other (2 timers, 2 quartzes, etc.). The tolerable difference between the two values must be determined and verified by the F-application. |

|                                                                                                                                                                                                                                                                                                                                     |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  <b>WARNING</b>                                                                                                                                                                                                                                    |
| <b>Permissible return values</b><br>From the point of view of the F-application, only the following return values are permitted: <ul style="list-style-type: none"><li>• RECV_FOUT_TELE_OK</li></ul> <b>Requirement: PSD_RQ_0_07</b><br>Impermissible return values have to induce the <b>HARD_FAIL</b> state of the F-application. |

#### 4.1.7 psd\_RecvFOutTele\_Invers(...)

|                                                                     |
|---------------------------------------------------------------------|
| <b>Note</b><br>Call this function cyclically in your F-application. |
|---------------------------------------------------------------------|

This function is to be used in the 1-channel variant. The function serves to transfer the inverse time values in a 1 ms grid and to generate the inverse data in the internal memory of the PSD for the specified instance.

|                  |                                                                                                           |
|------------------|-----------------------------------------------------------------------------------------------------------|
| <b>Function:</b> | <b>psd_RecvFOutTele_Invers(...)</b>                                                                       |
| Prototype:       | P_Byte psd_RecvFOutTele_Invers<br>(P_Word Inst_ID,<br>P_Word ClockValue_inv,<br>P_Word ClockValueC2_inv); |

| Transfer parameter: | Value range                            | Meaning                                        |
|---------------------|----------------------------------------|------------------------------------------------|
| Inst_ID             | INST_01...INST_X<br>X <= NOF_INSTANCES | Instance ID                                    |
| ClockValue          | 0...0xFFFF                             | Inverse 1 ms counter value<br>(1's complement) |
| ClockValueC2        | 0...0xFFFF                             | Inverse 1 ms counter value<br>(1's complement) |

| Return parameter: | Value range               | Meaning                  |
|-------------------|---------------------------|--------------------------|
| P_Byte            | RECV_FOUT_TELE_OK_INV     | Inverse data created     |
|                   | RECV_FOUT_TELE_NOT_OK_INV | Inconsistency recognized |

### WARNING

#### Permissible return values

From the point of view of the F-application, only the following return values are permitted:

- RECV\_FOUT\_TELE\_OK\_INV

#### Requirement: PSD\_RQ\_0\_07

Impermissible return values have to induce the **HARD\_FAIL** state of the F-application.

### WARNING

#### Requirement: PSD\_RQ\_1\_01

The return values of the used "inverse" functions are to be evaluated in accordance with the "normal" functions.

## 4.1.8 psd\_GetFOutData(...)

### Note

Call this function cyclically in your F-application.

Transfer of the F-output user data, the control byte identifiers and of the error status of the message frame check to the F-application for the specified instance.

| Function:  | psd_GetFOutData(...)                                                                                                                                                                |
|------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Prototype: | P_Byte psd_GetFOutData<br>(P_Word Inst_ID,<br>Fw_OutputDataCB P_GLOBDAT_ATTR_POINTER*<br>FwOutputDataCB_Ptr,<br>P_Byte P_GLOBDAT_ATTR_POINTER* OutDataArray,<br>P_Byte OutDataLen); |

| Transfer parameter: | Value range                           | Meaning                                                                                                |
|---------------------|---------------------------------------|--------------------------------------------------------------------------------------------------------|
| Inst_ID             | INST_01...INST_X<br>X ≤ NOF_INSTANCES | Instance ID                                                                                            |
| FwOutputDataCB_Ptr  | ---                                   | Pointer to a structure of the F-application for storing control byte identifiers and other information |

| Transfer parameter: | Value range     | Meaning                                                                       |
|---------------------|-----------------|-------------------------------------------------------------------------------|
| OutDataArray        | ---             | Pointer for a data area of the F-application for storing the output user data |
| OutDataLen          | 0..12 or 0..123 | Number of F-output user data bytes                                            |

| Return parameter: | Value range         | Meaning                                                                                                                                                                                                                     |
|-------------------|---------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| P_Byte            | F_OUTPUT_OK         | The toggle bit in the PROFIsafe control byte has toggled.                                                                                                                                                                   |
|                   | F_OUTPUT_OLD_CONSNR | The toggle bit in the PROFIsafe control byte has not toggled.                                                                                                                                                               |
|                   | F_OUTPUT_COMM_ERR   | An error has been detected in the PROFIsafe communication. The F-output user data are passivated. The control byte identifiers are invalid.                                                                                 |
|                   | F_OUTPUT_WD_TIMEOUT | A timeout was found in the communication. The F-output user data are passivated.                                                                                                                                            |
|                   | F_OUTPUT_PASSIVATED | The F-host has set Bit_4 (FV-to-be-activated) in the control byte to 1, or the PSD internal error counter that is decreased with each Good message frame is still greater than zero. The F-output user data are passivated. |
|                   | F_OUTPUT_NOT_OK     | Inconsistency found: <ul style="list-style-type: none"> <li>Transferred data length unequal to the data length calculated by the PSD</li> <li>Call not in PSD_DATAEX</li> </ul> The F-output user data are passivated.      |

### Note

To avoid using outdated values during the next transition from PSD\_PARAM to PSD\_DATAEX, the F-application should reset the control byte information in its buffer as follows after the call of **psd\_Stop()**:

- CB\_B0\_iParEN: F\_BIT\_FALSE
- CB\_B1\_OAReq: F\_BIT\_FALSE
- CB\_B4\_activate\_FV: F\_BIT\_FALSE
- CB\_B6\_ChF\_Ack: F\_BIT\_FALSE

**! WARNING****Requirement: PSD\_RQ\_0\_11**

The F-application must ensure the safe condition for the process for the following return values of **psd\_GetFOutData(...)**:

- F\_OUTPUT\_COMM\_ERR
- F\_OUTPUT\_WD\_TIMEOUT
- F\_OUTPUT\_PASSIVATED
- F\_OUTPUT\_NOT\_OK

**! WARNING****Permissible return values**

From the point of view of the F-application only the following return values are permitted:

- F\_OUTPUT\_OK
- F\_OUTPUT\_OLD\_CONSNR
- F\_OUTPUT\_COMM\_ERR
- F\_OUTPUT\_WD\_TIMEOUT
- F\_OUTPUT\_PASSIVATED

**Requirement: PSD\_RQ\_0\_07**

Impermissible return values have to induce the **HARD\_FAIL** state of the F-application.

| IDs from the PROFIsafe control byte and further information |                   |
|-------------------------------------------------------------|-------------------|
| typedef struct Fw_OutputDataCB_t {                          |                   |
| P_Byte                                                      | CB_B0_iParEN      |
| P_Byte                                                      | CB_B1_OAReq       |
| P_Byte                                                      | CB_B4_activate_FV |
| P_Byte                                                      | CB_B6_ChF_Ack     |
| P_Byte                                                      | TeleStatus_1      |
| P_Byte                                                      | TeleStatus_2      |
| P_Byte                                                      | TeleStatus_3      |
| } Fw_OutputDataCB;                                          |                   |

| Value range TeleStatus_1 | Meaning                                                                                                                                                                                                                                                                                 |
|--------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| F_OUTPUT_OK_TEL          | The PSD has received a fault-free message frame with a new or old sign-of-life. The F-application can output process values.                                                                                                                                                            |
| F_OUTPUT_NOT_OK_TEL      | The PSD has received a corrupt frame or a frame has been passivated by the F-host or by the PSD. The F-application must output fail-safe values. In "Last valid value" mode (optional operating mode of the F-application), the F-application outputs of the last valid process values. |

| Value range TeleStatus_2 | Meaning                                                                                                                                                         |
|--------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| F_OUTPUT_OK_NEW_TEL      | The PSD has received a fault-free message frame with a new sign-of-life. The F-application uses this information as a counter for frames with new sign-of-life. |
| F_OUTPUT_NOT_OK_NEW_TEL  | The PSD has received a corrupt frame and/or it is a frame with an old sign-of-life.                                                                             |

| Value range TeleStatus_3 | Meaning                                                                                                                                                                               |
|--------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| F_OUTPUT_COM_STARTED     | PROFIsafe communication has begun. The PSD has received a frame in which the toggle bit has gone from "0" to "1".                                                                     |
| F_OUTPUT_COM_NOT_STARTED | For the PSD, the PROFIsafe communication starts after calling <b>psd_Run(...)</b> only after the toggle bit is "1" in the control byte.<br>PROFIsafe communication has not yet begun. |

**WARNING****Requirement: PSD\_RQ\_0\_18**

The F-application has to take suitable measures in order to recognize different F-output user data or control byte identifiers between the two channels (for example due to an error in the output storage area). This can, for example, occur through comparison of the data transferred by the PSD. The test has to be carried before output to the process.

**WARNING****Requirement: PSD\_RQ\_0\_19**

If the PSD signals a communication error, the received control byte is invalid and the control byte identifiers transferred to the F-application CB\_B0\_iParEN, CB\_B1\_OAReq and CB\_B6\_ChF\_Ack are not to be evaluated.

## 4.1.9 psd\_GetFOutData\_Invers(...)

**Note**

Call this function cyclically in your F-application.

This function is to be used in the 1-channel variant. With this function the inverse F-output user data, the inverse control byte identifiers and the inverse error status of the specified instance are made available to the F-application.

| Function:  | psd_GetFOutData_Invers(...)                                                                                                                                                                     |
|------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Prototype: | <pre>P_Byte psd_GetFOutData_Invers (P_Word Inst_ID, Fw_OutputDataCB P_GLOBDAT_ATTR_POINTER* FFwOutputDataCB_Ptr_inv, P_Byte P_GLOBDAT_ATTR_POINTER* OutDataArray_inv, P_Byte OutDataLen);</pre> |

| Transfer parameter:     | Value range                           | Meaning                                                                                     |
|-------------------------|---------------------------------------|---------------------------------------------------------------------------------------------|
| Inst_ID                 | INST_01...INST_X<br>X ≤ NOF_INSTANCES | Instance ID                                                                                 |
| FwOutput-DataCB_Ptr_inv | ---                                   | Pointer to a structure for storing inverse control byte identifiers and further information |
| OutDataArray_inv        | ---                                   | Pointer to a data area for storing the inverse user data                                    |
| OutDataLen              | 0..12 or 0..123                       | Number of F-output user data bytes                                                          |

| Return parameter: | Value range             | Meaning                                                                                                                                                                                                                     |
|-------------------|-------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| P_Byte            | F_OUTPUT_OK_INV         | F-output message frame is valid. The F-output user data and control byte identifiers can be applied and evaluated by the F-application.                                                                                     |
|                   | F_OUTPUT_OLD_CONSNR_INV |                                                                                                                                                                                                                             |
|                   | F_OUTPUT_COMM_ERR_INV   | An error was found in the PROFIsafe communication. The F-output user data are passivated. The control byte identifiers are invalid.                                                                                         |
|                   | F_OUTPUT_WD_TIMEOUT_INV | A timeout was found in the communication. The F-output user data are passivated.                                                                                                                                            |
|                   | F_OUTPUT_PASSIVATED_INV | The F-host has set Bit_4 (FV-to-be-activated) in the control byte to 1, or the PSD internal error counter that is decreased with each Good message frame is still greater than zero. The F-output user data are passivated. |

**! WARNING**

**Permissible return values**

From the point of view of the F-application, only the following return values are permitted:

- F\_OUTPUT\_OK\_INV
- F\_OUTPUT\_OLD\_CONSNR\_INV
- F\_OUTPUT\_COMM\_ERR\_INV
- F\_OUTPUT\_WD\_TIMEOUT\_INV
- F\_OUTPUT\_PASSIVATED\_INV

**Requirement: PSD\_RQ\_0\_07**

Impermissible return values have to induce the **HARD\_FAIL** state of the F-application.

**Inverted identifiers from the PROFIsafe control byte and further information**

```
typedef struct Fw_OutputDataCB_t {
```

|        |                   |
|--------|-------------------|
| P_Byte | CB_B0_iParEN      |
| P_Byte | CB_B1_OAReq       |
| P_Byte | CB_B4_activate_FV |
| P_Byte | CB_B6_ChF_Ack     |
| P_Byte | TeleStatus_1      |
| P_Byte | TeleStatus_2      |
| P_Byte | TeleStatus_3      |

```
} Fw_OutputDataCB;
```

| Value range TeleStatus_1 | Meaning                                                                                                                                                                                                                                                                                 |
|--------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| F_OUTPUT_OK_TEL_INV      | The PSD has received a fault-free message frame with a new or old sign-of-life. The F-application can output process values.                                                                                                                                                            |
| F_OUTPUT_NOT_OK_TEL_INV  | The PSD has received a corrupt frame or a frame has been passivated by the F-host or by the PSD. The F-application must output fail-safe values. In "Last valid value" mode (optional operating mode of the F-application), the F-application outputs of the last valid process values. |

| Value range TeleStatus_2    | Meaning                                                                                                                                                         |
|-----------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| F_OUTPUT_OK_NEW_TEL_INV     | The PSD has received a fault-free message frame with a new sign-of-life. The F-application uses this information as a counter for frames with new sign-of-life. |
| F_OUTPUT_NOT_OK_NEW_TEL_INV | The PSD has received a corrupt frame and/or it is not a frame with an old sign-of-life.                                                                         |

| Value range TeleStatus_3     | Meaning                                                                                                                                                                            |
|------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| F_OUTPUT_COM_STARTED_INV     | PROFIsafe communication has begun. The PSD has received a frame in which the toggle bit has gone from "0" to "1".                                                                  |
| F_OUTPUT_COM_NOT_STARTED_INV | For the PSD, the PROFIsafe communication starts after calling <b>psd_Run(...)</b> only after the toggle bit is "1" in the control byte. PROFIsafe communication has not yet begun. |

**Note**

To avoid using outdated values during the next transition from PSD\_PARAM to PSD\_DATAEX, the F-application should reset the control byte information in its buffer as follows after the call of **psd\_Stop()**:

- CB\_B0\_iParEN: F\_BIT\_FALSE\_INV
- CB\_B1\_OAReq: F\_BIT\_FALSE\_INV
- CB\_B4\_activate\_FV: F\_BIT\_FALSE\_INV
- CB\_B6\_ChF\_Ack: F\_BIT\_FALSE\_INV

**WARNING****Requirement: PSD\_RQ\_1\_01**

The return values of the used "inverse" functions are to be evaluated in accordance with the "normal" functions.

**WARNING****Requirement: PSD\_RQ\_0\_18**

The F-application has to take suitable measures in order to recognize different F-output user data or control byte identifiers between the two channels (for example due to an error in the output storage area). This can, for example, occur through comparison of the data transferred by the PSD. The test has to be carried before output to the process.

**WARNING****Requirement: PSD\_RQ\_0\_19**

If the PSD signals a communication error, the received control byte is invalid and the control byte identifiers transferred to the F-application CB\_B0\_iParEN, CB\_B1\_OAReq and CB\_B6\_ChF\_Ack are not to be evaluated.

#### 4.1.10 psd\_SetFinData(...)

##### Note

Call this function cyclically in your F-application.

Acquisition of the status byte IDs and F-input user data of the specified instance by the F-application in a internal PSD buffer.

The acceptance to establish an F-input message frame is also carried out at a preceding communications error or timeout.

Acceptance begins with the cyclic data exchange.

| Function:  | psd_SetFinData(...)                                                                                                                                                                           |
|------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Prototype: | P_Byte psd_SetFinData<br>(P_Word Inst_ID,<br>const Fw_InputDataSB P_GLOBDAT_ATTR_POINTER*<br>FwInputDataSB_Ptr,<br>const P_Byte P_GLOBDAT_ATTR_POINTER*<br>InDataArray,<br>P_Byte InDataLen); |

| Transfer parameter: | Value range                           | Meaning                                                                                  |
|---------------------|---------------------------------------|------------------------------------------------------------------------------------------|
| Inst_ID             | INST_01...INST_X<br>X ≤ NOF_INSTANCES | Instance ID                                                                              |
| FwInDataSB_Ptr      | ---                                   | Pointer to a structure of the F-application for transferring the status byte identifiers |
| InDataArray         | ---                                   | Pointer for a data area of the F-application for storing the input user data             |
| InDataLen           | 0..12 or 0..123                       | Length of the input user data                                                            |

| Return parameter: | Value range         | Meaning                  |
|-------------------|---------------------|--------------------------|
| P_Byte            | SET_FIN_DATA_OK     | Data have been accepted. |
|                   | SET_FIN_DATA_NOT_OK | Inconsistency recognized |

**WARNING****Permissible return values**

From the point of view of the F-application, only the following return values are permitted:

- SET\_FIN\_DATA\_OK

**Requirement: PSD\_RQ\_0\_07**

Impermissible return values have to induce the **HARD\_FAIL** state of the F-application.

| IDs of the F-application for the construction of the PROFIsafe status byte |                   |
|----------------------------------------------------------------------------|-------------------|
| typedef struct Fw_InputDataSB_t {                                          |                   |
| P_Byte                                                                     | SB_B0_iParOk      |
| P_Byte                                                                     | SB_B1_DeviceFault |
| P_Byte                                                                     | SB_B4_FVactivated |
| } Fw_InputDataSB;                                                          |                   |

**Note**

The "SB\_B1\_DeviceFault" bit has the following meaning in accordance with the PROFIsafe profile:

- With F-parameter "F\_Passivation" = 0 (module-wide passivation): DeviceFault
- With F-parameter "F\_Passivation" = 1 (channel granular passivation): ChF\_Ack\_Req

#### 4.1.11 psd\_SetFinData\_Invers(...)

**Note**

Call this function cyclically in your F-application.

This function is to be used in the 1-channel variant. The function accepts the inverse F-input user data as well as status byte identifiers of the specified instance from the F-application into the internal buffer of the PSD.

| Function:  | psd_SetFinData_Invers(...)                                                                                                                                                                             |
|------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Prototype: | <pre>P_Byte psd_SetFinData_Invers (P_Word Inst_ID, const Fw_InputDataSB P_GLOBDAT_ATTR_POINTER* FFwInputDataSB_Ptr_inv, const P_Byte P_GLOBDAT_ATTR_POINTER* InDataArray_inv, P_Byte InDataLen);</pre> |

| Transfer parameter: | Value range                           | Meaning                                                                                          |
|---------------------|---------------------------------------|--------------------------------------------------------------------------------------------------|
| Inst_ID             | INST_01...INST_X<br>X ≤ NOF_INSTANCES | Instance ID                                                                                      |
| FFwInDataSB_Ptr_inv | ---                                   | Pointer to a structure of the F-application for transferring the inverse status byte identifiers |
| InDataArray_inv     | ---                                   | Pointer for a data area of the F-application for storing the inverse input user data             |
| InDataLen           | 0..12 or 0..123                       | Length of the input user data                                                                    |

| Return parameter: | Value range             | Meaning                  |
|-------------------|-------------------------|--------------------------|
| P_Byte            | SET_FIN_DATA_OK_INV     | Data have been accepted. |
|                   | SET_FIN_DATA_NOT_OK_INV | Inconsistency recognized |



#### WARNING

##### Permissible return values

From the point of view of the F-application, only the following return values are permitted:

- SET\_FIN\_DATA\_OK\_INV

##### Requirement: PSD\_RQ\_0\_07

Impermissible return values have to induce the **HARD\_FAIL** state of the F-application.



#### WARNING

##### Requirement: PSD\_RQ\_1\_01

The return values of the used "inverse" functions are to be evaluated in accordance with the "normal" functions.

| IDs of the F-application for the construction of the PROFIsafe status byte |                   |
|----------------------------------------------------------------------------|-------------------|
| typedef struct Fw_InputDataSB_t {                                          |                   |
| P_Byte                                                                     | SB_B0_iParOk      |
| P_Byte                                                                     | SB_B1_DeviceFault |
| P_Byte                                                                     | SB_B4_FVactivated |
| } Fw_InputDataSB;                                                          |                   |

4.1.12 **psd\_SendFlnTele(...)****Note**

Call this function cyclically in your F-application.

Forms the F-input frame from the F-input user data and status byte ID for the specified instance. The CRC value of the F-input message frame is used by the respective other channel.

Then transfer the F-input frame to the stack interface for sending to the F-host.

|                   |                                               |
|-------------------|-----------------------------------------------|
| <b>Function:</b>  | <b>psd_SendFlnTele(...)</b>                   |
| <b>Prototype:</b> | P_Byte psd_SendFlnTele(const P_Word Inst_ID); |

| <b>Transfer parameter:</b> | <b>Value range</b>                     | <b>Meaning</b> |
|----------------------------|----------------------------------------|----------------|
| Inst_ID                    | INST_01...INST_X<br>X <= NOF_INSTANCES | Instance ID    |

| <b>Return parameter:</b> | <b>Value range</b>   | <b>Meaning</b>                                       |
|--------------------------|----------------------|------------------------------------------------------|
| P_Byte                   | SEND_FIN_TELE_OK     | The F-input frame was passed to the stack interface. |
|                          | SEND_FIN_TELE_NOT_OK | Inconsistency recognized                             |

 **WARNING**
**Permissible return values**

From the point of view of the F-application, only the following return values are permitted:

- SEND\_FIN\_TELE\_OK

**Requirement: PSD\_RQ\_0\_07**

Impermissible return values have to induce the **HARD\_FAIL** state of the F-application.

#### 4.1.13 psd\_SendFlnTele\_Invers(...)

##### Note

Call this function cyclically in your F-application.

This function is to be used in the 1-channel variant. The function forms the inverse input message frame of the specified instance from the F-input user data and status byte identifiers transferred by the F-application.

|            |                                                |
|------------|------------------------------------------------|
| Function:  | psd_SendFlnTele_Invers(...)                    |
| Prototype: | P_Byte psd_SendFlnTele_Invers(P_Word Inst_ID); |

| Transfer parameter: | Value range                            | Meaning     |
|---------------------|----------------------------------------|-------------|
| Inst_ID             | INST_01...INST_X<br>X <= NOF_INSTANCES | Instance ID |

| Return parameter: | Value range              | Meaning                    |
|-------------------|--------------------------|----------------------------|
| P_Byte            | SEND_FIN_TELE_OK_INV     | F-input message frame sent |
|                   | SEND_FIN_TELE_NOT_OK_INV | Inconsistency recognized   |

##### WARNING

##### Permissible return values

From the point of view of the F-application, only the following return values are permitted:

- SEND\_FIN\_TELE\_OK\_INV

##### Requirement: PSD\_RQ\_0\_07

Impermissible return values have to induce the **HARD\_FAIL** state of the F-application.

##### WARNING

##### Requirement: PSD\_RQ\_1\_01

The return values of the used "inverse" functions are to be evaluated in accordance with the "normal" functions.

## 4.2 PSD interface variable `psd_SafetyVar`

The `psd_SafetyVar` interface variable enables the F-application to inform the PSD about a `HARD_FAIL`. Writing a value to `psd_SafetyVar` not equal to `P_SAFETY_CONST_OK` results in a CRC corruption when forming the F-input frame in PSD. This allows the F-host CRC error detection and thus a rapid response to the `HARD_FAIL` situation in the F-slave.



### WARNING

#### Requirement: `PSD_RQ_0_20`

If the F-application is in the `HARD_FAIL` state, the CRC of the F-input frame must be falsified in order for the F-host to faster initiate its measures to achieve the safe state of its application. It does not have to wait for the configured PROFIsafe timeout in this case.

The F-application can use the `psd_Falsify_InTelCRC(...)` function for this. No further PSD functions should be called.

## 4.3 Functions to be implemented by the F-application

### 4.3.1 `hard_err(...)`

In the case of an error the PSD carries out the following actions for all the instances:

- Set the PSD state to `PSD_HARD_FAIL`
- Set the instance-global safety variable `psd_SafetyVar` to `P_SAFETY_CONST_FALSE`
- Set the F-output user data of all the instances to 0.
- Falsify the CRC of the F-input message frame of all the instances

Then the PSD calls the `hard_err(...)` function from `p_c_fapplication.c` in order for the F-application to be able to react to the error. The passed parameters "Inst\_ID", "Line\_No." and "File\_Ident" can be used for diagnostics.

|            |                                                                                 |
|------------|---------------------------------------------------------------------------------|
| Function:  | <b><code>hard_err(...)</code></b>                                               |
| Prototype: | <code>void hard_err (P_Word Inst_ID, P_Word Line_No, P_Byte File_Ident);</code> |

| Transfer parameter: | Value range                           | Meaning                                         |
|---------------------|---------------------------------------|-------------------------------------------------|
| Inst_ID             | INST_01...INST_X<br>X ≤ NOF_INSTANCES | Instance ID                                     |
| Line_No             | 0...0xFFFF                            | Line no.                                        |
| File_Ident          | 0...0xFF                              | File_ID is defined in <code>p_c_config.h</code> |

| Return parameter: | Value range | Meaning |
|-------------------|-------------|---------|
| none              | ---         | ---     |

#### WARNING

##### Requirement: PSD\_RQ\_0\_08

The **hard\_err()** function must be implemented by the F-application. It is called by the PSD when it goes into the **PSD\_HARD\_FAIL** state (Fig. 2-4). The function must result in a safe state for all used instances. No further PSD functions need be called within **hard\_err()**.

#### WARNING

##### Requirement: PSD\_RQ\_0\_15

All interface functions except **hard\_err(...)** must return to PSD.

### 4.3.2 psd\_GetFParPtr(...)

The prototype of the function is contained in the file p\_c\_si.h and has to be implemented by the F-application. It makes a pointer to the F-parameters of the specified instance available to the PSD.

|            |                                          |
|------------|------------------------------------------|
| Function:  | psd_GetFParPtr(...)                      |
| Prototype: | P_Byte* psd_GetFParPtr (P_Word Inst_ID); |

| Transfer parameter: | Value range                             | Meaning     |
|---------------------|-----------------------------------------|-------------|
| Inst_ID             | INST_01 ... INST_X<br>X ≤ NOF_INSTANCES | Instance ID |

| Return parameter: | Value range | Meaning                                                                                                              |
|-------------------|-------------|----------------------------------------------------------------------------------------------------------------------|
| P_Byte*           | ---         | Pointer to the buffer for the F-parameters of the instance. A zero pointer means a Hard Error situation for the PSD. |

### 4.3.3 psd\_GetConfigPtr(...)

The prototype of the function is contained in the file p\_c\_si.h and has to be implemented by the F-application. It provides the PSD a pointer to the structure with the frame lengths (consisting of user data + PROFIsafe trailer) of the specified instance. For the layout of the structure for the I/O frame lengths, see section psd\_Config(...) (Page 34). The PSD determines the resulting user data lengths in accordance with its parameterization.

|                   |                                            |
|-------------------|--------------------------------------------|
| <b>Function:</b>  | <b>psd_GetConfigPtr(...)</b>               |
| <b>Prototype:</b> | P_Byte* psd_GetConfigPtr (P_Word Inst_ID); |

| <b>Transfer parameter:</b> | <b>Value range</b>                       | <b>Meaning</b> |
|----------------------------|------------------------------------------|----------------|
| Inst_ID                    | INST_01 ... INST_X<br>X <= NOF_INSTANCES | Instance ID    |

| <b>Return parameter:</b> | <b>Value range</b> | <b>Meaning</b>                                                                                                            |
|--------------------------|--------------------|---------------------------------------------------------------------------------------------------------------------------|
| P_Byte*                  | ---                | Pointer to the structure with the frame lengths of the instance. A zero pointer means a Hard Error situation for the PSD. |

### 4.3.4 psd\_GetOutputTelegram(...)

The prototype of the function is contained in the file p\_c\_si.h and has to be implemented by the F-application. It provides the PSD a pointer to the F-output frame from the F-host for the specified instance.

|                   |                                                  |
|-------------------|--------------------------------------------------|
| <b>Function:</b>  | <b>psd_GetOutput_Telegram(...)</b>               |
| <b>Prototype:</b> | P_Byte* psd_GetOutput_Telegram (P_Word Inst_ID); |

| <b>Transfer parameter:</b> | <b>Value range</b>                       | <b>Meaning</b> |
|----------------------------|------------------------------------------|----------------|
| Inst_ID                    | INST_01 ... INST_X<br>X <= NOF_INSTANCES | Instance ID    |

| <b>Return parameter:</b> | <b>Value range</b> | <b>Meaning</b>                                                                                                         |
|--------------------------|--------------------|------------------------------------------------------------------------------------------------------------------------|
| P_Byte*                  | ---                | Pointer to the buffer for the F-output frame of the instance. A zero pointer means a Hard Error situation for the PSD. |

### 4.3.5 psd\_GetInputBuffer(...)

The prototype of the function is contained in the file p\_c\_si.h and has to be implemented by the F-application. It provides the PSD a pointer to the F-input frame for the specified instance. The F-input message frame consists of the F-input user data + PROFIsafe Trailer.

|                  |                                                                       |
|------------------|-----------------------------------------------------------------------|
| <b>Function:</b> | <b>psd_GetInputBuffer(...)</b>                                        |
| Prototype:       | P_Byte* psd_GetInputBuffer<br>(P_Word Inst_ID,<br>P_Byte* InTeleLen); |

| Transfer parameter: | Value range                              | Meaning                                  |
|---------------------|------------------------------------------|------------------------------------------|
| Inst_ID             | INST_01 ... INST_X<br>X <= NOF_INSTANCES | Instance ID                              |
| InTeleLen           | 0...12/123 bytes                         | Length of the F-input message frame area |

| Return parameter: | Value range | Meaning                                                                                                               |
|-------------------|-------------|-----------------------------------------------------------------------------------------------------------------------|
| P_Byte*           | ---         | Pointer to the buffer for the F-input frame of the instance. A zero pointer means a Hard Error situation for the PSD. |

### 4.3.6 psd\_InputSendAck(...)

The prototype of the function is contained in the file p\_c\_si.h and has to be implemented by the F-application. The function signals the stack interface that the F-input frame from the PSD is filled and ready to be sent to the F-host.

|                  |                                           |
|------------------|-------------------------------------------|
| <b>Function:</b> | <b>psd_InputSendAck(...)</b>              |
| Prototype:       | P_Byte psd_InputSendAck (P_Word Inst_ID); |

| Transfer parameter: | Value range                              | Meaning     |
|---------------------|------------------------------------------|-------------|
| Inst_ID             | INST_01 ... INST_X<br>X <= NOF_INSTANCES | Instance ID |

| Return parameter: | Value range         | Meaning                                                             |
|-------------------|---------------------|---------------------------------------------------------------------|
| P_Byte            | FIN_TELE_UPDATE_OK  | The input frame was made available to the F-host.                   |
|                   | FIN_TELE_UPDATE_ERR | Error acquiring the input frame (Hard Error situation for the PSD). |

#### 4.3.7 enable\_int(...)

The prototype of the function is contained in the file p\_c\_fapplication.h and must be implemented by the F-application. The PSD signals that the interrupt disable for this instance should be released.

|                   |                                   |
|-------------------|-----------------------------------|
| <b>Function:</b>  | <b>Enable_int(...)</b>            |
| <b>Prototype:</b> | void enable_int (P_Word Inst_ID); |

| <b>Transfer parameter:</b> | <b>Value range</b>                       | <b>Meaning</b> |
|----------------------------|------------------------------------------|----------------|
| Inst_ID                    | INST_01 ... INST_X<br>X <= NOF_INSTANCES | Instance ID    |

| <b>Return parameter:</b> | <b>Value range</b> | <b>Meaning</b> |
|--------------------------|--------------------|----------------|
| none                     | ---                | ---            |

#### 4.3.8 disable\_int(...)

The prototype of the function is contained in the file p\_c\_fapplication.h and must be implemented by the F-application. The PSD requests an interrupt disable for the specified instance because the PSD must access data with consistence.

|                   |                                    |
|-------------------|------------------------------------|
| <b>Function:</b>  | <b>Disable_int(...)</b>            |
| <b>Prototype:</b> | void disable_int (P_Word Inst_ID); |

| <b>Transfer parameter:</b> | <b>Value range</b>                       | <b>Meaning</b> |
|----------------------------|------------------------------------------|----------------|
| Inst_ID                    | INST_01 ... INST_X<br>X <= NOF_INSTANCES | Instance ID    |

| <b>Return parameter:</b> | <b>Value range</b> | <b>Meaning</b> |
|--------------------------|--------------------|----------------|
| none                     | ---                | ---            |

## 4.4 Functions to be implemented at the 2-channel generation variant

In the 2-channel generation variant, the PSD requires a synchronization interface to synchronize the data.

The prototypes of the functions are defined in the `p_c_fapplication.h` file and allow the PSD to access the synchronization interface provided by the F- application. The `p_c_fapplication.c` is available for customizing.

### Note

For an F-slave that uses different byte ordering for the two channels, this must be taken into account for each synchronized value.



### WARNING

#### Requirement: PSD\_RQ\_2\_01

The F-application must provide a synchronization interface with time monitoring for 2-channel PSD generation. Mirroring of the data to be exchanged is to be excluded.

### 4.4.1 psd\_OutTransfer(...)

This function is called by the PSD in the `psd_RecvFOutTele(...)` function. The PSD passes the current value of program flow control for an instance to the synchronization interface. The function initiates the transfer to the second channel, so that it can compare the value to its own value using the `psd_OutSync(...)` function at a later point in time.

|                  |                                                                  |
|------------------|------------------------------------------------------------------|
| <b>Function:</b> | <b>psd_OutTransfer(...)</b>                                      |
| Prototype:       | P_Byte psd_OutTransfer<br>(P_Word Inst_ID,<br>P_Word FlowValue); |

| Transfer parameter: | Value range                             | Meaning                                   |
|---------------------|-----------------------------------------|-------------------------------------------|
| Inst_ID             | INST_01 ... INST_X<br>X ≤ NOF_INSTANCES | Instance ID                               |
| FlowValue           | 0...0xFFFF                              | Current value of the program flow control |

| Return parameter: | Value range     | Meaning                                             |
|-------------------|-----------------|-----------------------------------------------------|
| P_Byte            | TRANSFER_OK     | Transfer OK                                         |
|                   | TRANSFER_NOT_OK | Transfer not OK (Hard Error situation for the PSD). |

### 4.4.2 psd\_OutSync(...)

This function is called by the PSD in the **psd\_GetFOutData(...)** function. The PSD uses this function to fetch synchronization data (program flow control from the other channel) received in the meantime. If the data have not yet arrived, the function waits for the arrival. The PSD then compares the thus received data with the data of its own channel. This means that the synchronization data from the other channel must be available "in good time", i.e. within a device-specific synchronization timeout. Different synchronization data send the PSD to the PSD\_HARD\_FAIL state.

It must be ensured that the buffered data are respectively read only once (for example, by clearing the buffer after output).

|                   |                                                                |
|-------------------|----------------------------------------------------------------|
| <b>Function:</b>  | <b>psd_OutSync(...)</b>                                        |
| <b>Prototype:</b> | P_Byte psd_OutSync<br>(P_Word Inst_ID,<br>P_Word * FlowValue); |

| <b>Transfer parameter:</b> | <b>Value range</b>                       | <b>Meaning</b>                                                                                                                 |
|----------------------------|------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| Inst_ID                    | INST_01 ... INST_X<br>X <= NOF_INSTANCES | Instance ID                                                                                                                    |
| P_Word *FlowValue          | ---                                      | Pointer to the memory in which the function enters the synchronization data (program flow control value) of the other channel. |

| <b>Return parameter:</b> | <b>Value range</b> | <b>Meaning</b>                                            |
|--------------------------|--------------------|-----------------------------------------------------------|
| P_Byte                   | SYNC_OK            | Synchronization OK                                        |
|                          | SYNC_NOT_OK        | Synchronization not OK<br>(Hard error situation for PSD). |

### 4.4.3 psd\_InTransfer(...)

This function is called by the PSD in the **psd\_SetFInData(...)** function. The PSD passes the CRC of the F-input frame for an instance to the synchronization interface. The function initiates the transfer to the second channel, so that it can access the CRC of the F-input frame using the **psd\_InSync(...)** function at a later point in time.

|                  |                                                                 |
|------------------|-----------------------------------------------------------------|
| <b>Function:</b> | <b>psd_InTransfer(...)</b>                                      |
| Prototype:       | P_Byte psd_InTransfer<br>(P_Word Inst_ID,<br>P_DWord CRCValue); |

| Transfer parameter: | Value range                              | Meaning                                                  |
|---------------------|------------------------------------------|----------------------------------------------------------|
| Inst_ID             | INST_01 ... INST_X<br>X <= NOF_INSTANCES | Instance ID                                              |
| CRCValue            | 0...0xFFFFFFFF                           | CRC value of the F-input frame that is to be transferred |

| Return parameter: | Value range     | Meaning                                             |
|-------------------|-----------------|-----------------------------------------------------|
| P_Byte            | TRANSFER_OK     | Transfer OK                                         |
|                   | TRANSFER_NOT_OK | Transfer not OK (Hard Error situation for the PSD). |

#### 4.4.4 psd\_InSync(...)

This function is called by the PSD in the **psd\_SendFInTele(...)** function. The PSD uses this function to fetch synchronization data (CRC value from the other channel) received in the meantime. If the data have not yet arrived, the function waits for the arrival. The PSD then compares the thus received data with the data of its own channel. This means that the synchronization data from the other channel must be available "in good time", i.e. within a device-specific synchronization timeout. Different synchronization data send the PSD to the PSD\_HARD\_FAIL state.

It must be ensured that the buffered data are respectively read only once (for example, by clearing the buffer after output).

|                   |                                                              |
|-------------------|--------------------------------------------------------------|
| <b>Function:</b>  | <b>psd_InSync(...)</b>                                       |
| <b>Prototype:</b> | P_Byte psd_InSync<br>(P_Word Inst_ID,<br>P_DWord* CRCValue); |

| <b>Transfer parameter:</b> | <b>Value range</b>                       | <b>Meaning</b>                                                                                                                     |
|----------------------------|------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|
| Inst_ID                    | INST_01 ... INST_X<br>X <= NOF_INSTANCES | Instance ID                                                                                                                        |
| P_DWord *CrcValue          | 0...0xFFFFFFFF                           | Pointer to the memory in which the function enters the synchronization data (CRC value of the F-input frame) of the other channel. |

| <b>Return parameter:</b> | <b>Value range</b> | <b>Meaning</b>                                             |
|--------------------------|--------------------|------------------------------------------------------------|
| P_Byte                   | SYNC_OK            | Synchronization OK                                         |
|                          | SYNC_NOT_OK        | Synchronization not OK (Hard Error situation for the PSD). |

## 4.5 Optional PSD interface functions



### WARNING

Requirement: PSD\_RQ\_0\_15

All interface functions except **hard\_err(...)** must return to PSD.

### 4.5.1 Auxiliary functions for the CRC calculation

#### 4.5.1.1 psd\_CRC16(...)

The function calculates a 16-bit CRC value over a data area. In the process the generator polynomial 0x14EAB is used.

|                   |                                                                                                             |
|-------------------|-------------------------------------------------------------------------------------------------------------|
| <b>Function:</b>  | <b>psd_CRC16(...)</b>                                                                                       |
| <b>Prototype:</b> | P_Word psd_CRC16<br>(P_DWord Length, P_Word StartValue,<br>const P_Byte Data_Ptr[ ], P_Byte CalcDirection); |

| Transfer parameter: | Value range    | Meaning                                                                                                                                                             |
|---------------------|----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Length              | 0 - 0xFFFFFFFF | Length of the data for the calculation                                                                                                                              |
| StartValue          | 0 - 0xFFFF     | Start CRC16 value for the calculation                                                                                                                               |
| Data_Ptr            | ---            | Pointer to the data for calculation                                                                                                                                 |
| CalcDirection       | FORWARD        | Calculation is carried out from the start address of the data of the CRC16 values across the values in accordance with the length specification in ascending order. |
|                     | BACKWARD       | Calculation is carried out from the end address of the data of the CRC16 values across the values in accordance with the length specification in descending order.  |

| Return parameter: | Value range | Meaning          |
|-------------------|-------------|------------------|
| P_Word            | 0 – 0xFFFF  | 16-bit CRC value |

## 4.5.1.2 psd\_CRC24(...)

The function calculates a 24-bit CRC value over a data area. In the process the generator polynomial 0x5D6DCB is used.

|                   |                                                                                                               |
|-------------------|---------------------------------------------------------------------------------------------------------------|
| <b>Function:</b>  | <b>psd_CRC24(...)</b>                                                                                         |
| <b>Prototype:</b> | P_DWord psd_CRC24<br>(P_DWord Length, P_DWord StartValue,<br>const P_Byte Data_Ptr[ ], P_Byte CalcDirection); |

| <b>Transfer parameter:</b> | <b>Value range</b> | <b>Meaning</b>                                                                                                                                                      |
|----------------------------|--------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Length                     | 0 - 0xFFFFFFFF     | Length of the data for the calculation                                                                                                                              |
| StartValue                 | 0 - 0x00FFFFFF     | Start CRC24 value for the calculation                                                                                                                               |
| Data_Ptr                   | ---                | Pointer to the data for calculation                                                                                                                                 |
| CalcDirection              | FORWARD            | Calculation is carried out from the start address of the data of the CRC24 values across the values in accordance with the length specification in ascending order. |
|                            | BACKWARD           | Calculation is carried out from the end address of the data of the CRC24 values across the values in accordance with the length specification in descending order.  |

| <b>Return parameter:</b> | <b>Value range</b> | <b>Meaning</b>   |
|--------------------------|--------------------|------------------|
| P_DWord                  | 0 – 0x00FFFFFF     | 24-bit CRC value |

### 4.5.1.3 psd\_CRC32(...)

The function calculates a 32-bit CRC value over a data area. In the process the generator polynomial 0x1F4ACFB13 is used.

|                   |                                                                                                               |
|-------------------|---------------------------------------------------------------------------------------------------------------|
| <b>Function:</b>  | <b>psd_CRC32(...)</b>                                                                                         |
| <b>Prototype:</b> | P_DWord psd_CRC32<br>(P_DWord Length, P_DWord StartValue,<br>const P_Byte Data_Ptr[ ], P_Byte CalcDirection); |

| Transfer parameter: | Value range    | Meaning                                                                                                                                                             |
|---------------------|----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Length              | 0 - 0xFFFFFFFF | Length of the data for the calculation                                                                                                                              |
| StartValue          | 0 - 0xFFFFFFFF | Start CRC32 value for the calculation                                                                                                                               |
| Data_Ptr            | ---            | Pointer to the data for calculation                                                                                                                                 |
| CalcDirection       | FORWARD        | Calculation is carried out from the start address of the data of the CRC32 values across the values in accordance with the length specification in ascending order. |
|                     | BACKWARD       | Calculation is carried out from the end address of the data of the CRC32 values across the values in accordance with the length specification in descending order.  |

| Return parameter: | Value range    | Meaning          |
|-------------------|----------------|------------------|
| P_DWord           | 0 – 0xFFFFFFFF | 32-bit CRC value |

### 4.5.2 psd\_GetState(...)

You can query the current PSD state with the function.

|                   |                                      |
|-------------------|--------------------------------------|
| <b>Function:</b>  | <b>psd_GetState(...)</b>             |
| <b>Prototype:</b> | P_Byte psd_GetState(P_Word Inst_ID); |

| Transfer parameter: | Value range                            | Meaning     |
|---------------------|----------------------------------------|-------------|
| Inst_ID             | INST_01...INST_X<br>X <= NOF_INSTANCES | Instance ID |

| Return parameter: | Value range | Meaning                     |
|-------------------|-------------|-----------------------------|
| P_Byte            | PSD_INIT    | PSD initialized             |
|                   | PSD_PARAM   | PSD parameterized           |
|                   | PSD_DATAEX  | PSD in cyclic data exchange |

### 4.5.3 psd\_GetFPar(...)

You can use this function to read the following F-parameters:

- F\_iPar\_CRC
- SIL value
- Type of passivation: module or channel granular
- F\_WD\_Time

The function can be called in the PSD states PSD\_PARAM or PSD\_DATAEX.

|                   |                                                                |
|-------------------|----------------------------------------------------------------|
| <b>Function:</b>  | <b>psd_GetFPar(...)</b>                                        |
| <b>Prototype:</b> | P_Byte psd_GetFPar(P_Word Inst_ID<br>Fw_PsdFPar* FwFParamPtr); |

| <b>Transfer parameter:</b> | <b>Value range</b>                     | <b>Meaning</b>                        |
|----------------------------|----------------------------------------|---------------------------------------|
| Inst_ID                    | INST_01...INST_X<br>X <= NOF_INSTANCES | Instance ID                           |
| FwFParamPtr                | ---                                    | Pointer to the F-parameter structure. |

| <b>Return parameter:</b> | <b>Value range</b> | <b>Meaning</b>                                                                                                                                                                                 |
|--------------------------|--------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| P_Byte                   | GET_FPAR_OK        | Call correct                                                                                                                                                                                   |
|                          | GET_FPAR_NOT_OK    | <ul style="list-style-type: none"> <li>• Pointer to the F-parameter structure is 0</li> </ul> or <ul style="list-style-type: none"> <li>• PSD state is unequal PSD_PARAM/PSD_DATAEX</li> </ul> |

**Layout of the device parameter structure:**

|                                                                                                                                                                                                                                                                                                                                                                                                    | Meaning / range of values                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> #ifdef REDUNDANT typedef struct Fw_PsdFPar_t {     P_DWord FiParCRC     P_Word FWdTime     P_Byte FSil     P_Byte FPassivation } Fw_PsdFPar; #else typedef struct Fw_PsdFPar_t {     P_DWord FiParCRC     P_DWord FiParCRC_inv     P_Word FWdTime     P_Word FWdTime_inv     P_Byte FSil     P_Byte FSil_inv     P_Byte FPassivation     P_Byte FPassivation_inv } Fw_PsdFPar; #endif </pre> | <ul style="list-style-type: none"> <li>FiParCRC: FiParCRC (<math>\geq 0</math> if not contained in the F-parameters)</li> <li>FiParCRC_inv: FiParCRC inverted (<math>\leq 0xFFFFFFFF</math> if not contained in the F parameters)</li> <li>FWdTime: <math>0xFFFFE - 0</math></li> <li>FWdTime_inv: <math>1 - 0xFFFF</math></li> <li>FSil: SIL_1, SIL_2, SIL_3, NO_SIL</li> <li>FSil_inv: SIL_1_INV, SIL_2_INV, SIL_3_INV, NO_SIL_INV</li> <li>FPassivation: CHANNEL_GRANULAR_PASSIVATION, F_MODULE_PASSIVATION</li> <li>FPassivation_inv: CHANNEL_GRANULAR_PASSIVATION_INV, F_MODULE_PASSIVATION_INV</li> </ul> |

**4.5.4 psd\_GetConfig(...)**

You can use this function to read the following configuration parameters:

- Length of the F-input user data
- Length of the F-output user data

The function can be called in the PSD states PSD\_PARAM (following length configuration screen) or PSD\_DATAEX.

|                   |                                                                  |
|-------------------|------------------------------------------------------------------|
| <b>Function:</b>  | <b>psd_GetConfig(...)</b>                                        |
| <b>Prototype:</b> | P_Byte psd_GetConfig(P_Word Inst_ID, Fw_PsdConfig* FwConfigPtr); |

| Transfer parameter: | Value range                                        | Meaning                                           |
|---------------------|----------------------------------------------------|---------------------------------------------------|
| Inst_ID             | INST_01...INST_X<br>$X \leq \text{NOF\_INSTANCES}$ | Instance ID                                       |
| FwConfigPtr         | ---                                                | Pointer to the configuration parameter structure. |

| Return parameter: | Value range       | Meaning                                            |
|-------------------|-------------------|----------------------------------------------------|
| P_Byte            | GET_CONFIG_OK     | Call correct                                       |
|                   | GET_CONFIG_NOT_OK | Pointer to configuration parameter structure is 0. |

**Layout of the device parameter structure:**

|                                                                                                                                                                                                                                                                                                                                                                                                        | Meaning                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> #ifdef REDUNDANT typedef struct Fw_PsdConfig_t {     P_Byte FInput_Data_Len     P_Byte FOutput_Data_Len     P_Byte TrailerLength } Fw_PsdConfig; #else typedef struct Fw_PsdConfig_t {     P_Byte FInput_Data_Len     P_Byte FInput_Data_Len_inv     P_Byte FOutput_Data_Len     P_Byte FOutput_Data_Len_inv     P_Byte TrailerLength     P_Byte TrailerLength_inv } Fw_PsdConfig; #endif </pre> | <ul style="list-style-type: none"> <li>FInput_Data_Len: Length of the F-input user data</li> <li>FInput_Data_Len_inv: Inverted length of the F-input user data</li> <li>FOutput_Data_Len: Length of the F-output user data</li> <li>FOutput_Data_Len_inv: Inverted length of the F-output user data</li> <li>P_Byte TrailerLength: Length of the PROFIsafe trailer</li> <li>P_Byte TrailerLength_inv: Inverted length of the PROFIsafe trailer</li> </ul> |

**4.5.5 psd\_GetPSD\_Version(...)**

You can use this function to read the current version of the PSD.

|                   |                                                      |
|-------------------|------------------------------------------------------|
| <b>Function:</b>  | <b>psd_GetPSD_Version(...)</b>                       |
| <b>Prototype:</b> | void psd_GetPSD_Version(Fw_PSD_Version *VersionPtr); |

| Transfer parameter: | Value range | Meaning                                                                                              |
|---------------------|-------------|------------------------------------------------------------------------------------------------------|
| VersionPtr          | ---         | Pointer to a structure for the PSD version. A zero pointer means a Hard Error situation for the PSD. |

| Return parameter: | Value range | Meaning |
|-------------------|-------------|---------|
| ---               | ---         | ---     |

**Design of the PSD version structure:**

|                                   | Meaning / range of values                                                                                                                                                                                                                   |
|-----------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| typedef struct Fw_PSD_Version_t { |                                                                                                                                                                                                                                             |
| P_Byte V_type                     | The values defined in p_global.h are output: <ul style="list-style-type: none"> <li>V_type: Value of VERSION_TYPE</li> <li>V_no1: Value of VERSION_NO1</li> <li>V_no2: Value of VERSION_NO2</li> <li>V_no3: Value of VERSION_NO3</li> </ul> |
| P_Byte V_no1                      |                                                                                                                                                                                                                                             |
| P_Byte V_no2                      |                                                                                                                                                                                                                                             |
| P_Byte V_no3                      |                                                                                                                                                                                                                                             |
| } Fw_PSD_Version;                 |                                                                                                                                                                                                                                             |

#### 4.5.6 psd\_Falsify\_InTelCRC(...)

If the F-application is in the **HARD\_FAIL** state, the CRC of the F-input frame must be falsified. The F-application, which does not necessarily know the structure of the F-input frame, uses the **psd\_Falsify\_InTelCRC(...)** function for this.

The function request a buffer for the F-input frame for a communication instance, enters a bad CRC and makes the buffer available for sending to the F-host. The function can only be performed after successful execution of **psd\_Config(...)**, because that is when the structure of the F-input frame is first defined.

|            |                                                    |
|------------|----------------------------------------------------|
| Function:  | <b>psd_Falsify_FInTelCRC(...)</b>                  |
| Prototype: | P_Byte psd_Falsify_InTelCRC(const P_Word Inst_ID); |

| Return parameter: | Value range               | Meaning                                                                                              |
|-------------------|---------------------------|------------------------------------------------------------------------------------------------------|
| P_Byte            | FALSIFY_IN_TEL_CRC_OK     | CRC falsification performed                                                                          |
|                   | FALSIFY_IN_TEL_CRC_NOT_OK | Pointer to F-input frame invalid or function was called before execution of <b>psd_Config(...)</b> . |



#### **WARNING**

##### **Requirement: PSD\_RQ\_0\_20**

If the F-application is in the **HARD\_FAIL** state, the CRC of the F-input frame must be falsified in order for the F-host to faster initiate its measures to achieve the safe state of its application. It does not have to wait for the configured PROFIsafe timeout in this case.

The F-application can use the **psd\_Falsify\_InTelCRC(...)** function for this. No further PSD functions should be called.

### 4.5.7 psd\_GetTraceData(...)

Systematic CRC errors can be caused by standard components during operation. A CRC error can, for example, be caused by the infrastructure (inconsistent frames).

Additional data are required for analysis to diagnose a CRC error in the PROFIsafe frame in the field.

If the PSD recognizes a CRC error, it saves certain diagnostics data once in its instance data. The interface function `psd_GetFOutData(..)` shows the CRC error with its return value to the F-application (section "`psd_GetFOutData(...)` (Page 40)". The interface function `psd_GetTraceData(...)` enables the F-application to read out the diagnostics data and make it available to the user.

The diagnostics data are updated by the PSD only after the CRC error has gone and a new CRC error is identified by the PSD.

|                   |                                                                             |
|-------------------|-----------------------------------------------------------------------------|
| <b>Function:</b>  | <b>psd_GetTraceData(...)</b>                                                |
| <b>Prototype:</b> | P_Byte psd_GetTraceData (P_Word Inst_ID, Fw_PsdTraceData* FwTraceData_Ptr); |

| Transfer parameter: | Value range                                     | Meaning                                                                  |
|---------------------|-------------------------------------------------|--------------------------------------------------------------------------|
| Inst_ID             | INST_01...INST_X<br>X <= NOF_INSTANCES          | Instance ID                                                              |
| FwTraceData_Ptr     | See the structure of the diagnostics data below | Pointer to a structure of the F-application for storing diagnostics data |

| Return parameter: | Value range           | Meaning                                                            |
|-------------------|-----------------------|--------------------------------------------------------------------|
| P_Byte            | GET_TRACE_DATA_OK     | Diagnostics data saved                                             |
|                   | GET_TRACE_DATA_NOT_OK | Inst_ID invalid or FwTraceData_Ptr = 0. Diagnostics data not saved |

#### Structure of the diagnostics data:

| Code                              | Meaning                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|-----------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| typedef struct Fw_PsdTraceData {  | <ul style="list-style-type: none"> <li>Version: Version of the diagnostics data</li> <li>IncNr_1/2: used in V2.6.1 mode</li> <li>ReceivedTelegramCRC: CRC of the received F-output frame with CRC error.</li> <li>CalculatedTelegramCRC: CRC of the F-output frame calculated by PSD</li> <li>VirtualConsecutiveNr: used in V2.4 mode</li> <li>Codename: Code name used in V2.6.1 mode</li> <li>Modifier: Modifier used in V2.6.1 mode</li> <li>FiParCRC: CRC of the iParameters</li> <li>FParameterCRC: CRC of the F-parameters</li> </ul> |
| P_Word      Version               |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| P_QWord     IncNr_1               |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| P_QWord     IncNr_2               |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| P_DWord     ReceivedTelegramCRC   |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| P_DWord     CalculatedTelegramCRC |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| P_DWord     VirtualConsecutiveNr  |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| P_DWord     Codename              |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| P_DWord     Modifier              |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| P_DWord     FiParCRC              |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| P_Word      FParameterCRC         |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| } Fw_PsdTraceData;                |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |

## 4.5.8 Inverse PSD functions

In the 1-channel generation variant the PSD makes a corresponding inverse function available for each main function. The functions described below support a 1-channel software architecture of the F-application. The functions return the results of the inverse calculation path of the PSD. The meaning of the return values is specified in the main functions.



### WARNING

#### Requirement: PSD\_RQ\_1\_01

The return values of the used "inverse" functions are to be evaluated in accordance with the "normal" functions.

### 4.5.8.1 psd\_InitInstance\_Invers(...)

Functionality corresponds to the normal function psd\_InitInstance(...) (Page 30).

|            |                                                  |
|------------|--------------------------------------------------|
| Function:  | psd_InitInstance_Invers(...)                     |
| Prototype: | P_Byte psd_InitInstance_Invers (P_Word Inst_ID); |

| Transfer parameter: | Value range                           | Meaning     |
|---------------------|---------------------------------------|-------------|
| Inst_ID             | INST_01...INST_X<br>X ≤ NOF_INSTANCES | Instance ID |

| Return parameter: | Value range          | Meaning                   |
|-------------------|----------------------|---------------------------|
| P_Byte            | INIT_INSTANCE_OK_INV | Initialization successful |

## 4.5.8.2 psd\_FParBuild\_Invers(...)

Functionality corresponds to the normal function psd\_FParBuild(...) (Page 31).

|                   |                                               |
|-------------------|-----------------------------------------------|
| <b>Function:</b>  | <b>psd_FParBuild_Invers(...)</b>              |
| <b>Prototype:</b> | P_Byte psd_FParBuild_Invers (P_Word Inst_ID); |

| <b>Transfer parameter:</b> | <b>Value range</b>                     | <b>Meaning</b> |
|----------------------------|----------------------------------------|----------------|
| Inst_ID                    | INST_01...INST_0X<br>X ≤ NOF_INSTANCES | Instance ID    |

| <b>Return parameter:</b> | <b>Value range</b>                        | <b>Meaning</b>                                                                                                                                           |
|--------------------------|-------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| P_Byte                   | FPAR_BUILD_OK_INV                         | F-parameter OK                                                                                                                                           |
|                          | FPAR_F_DEST_ADD_MISMATCH_INV              | Incorrect target address                                                                                                                                 |
|                          | FPAR_F_DEST_ADD_NOT_VALID_INV             | Invalid target address                                                                                                                                   |
|                          | FPAR_F_SOURCE_ADD_MISMATCH_I<br>NV        | Incorrect source address                                                                                                                                 |
|                          | FPAR_F_SOURCE_ADD_NOT_VALID_I<br>NV       | Invalid source address                                                                                                                                   |
|                          | FPAR_F_WD_TIME_NULL_INV                   | Watchdog time is invalid (0)                                                                                                                             |
|                          | FPAR_F_SIL_ERR_INV                        | Configured SIL class too high                                                                                                                            |
|                          | FPAR_CRC_LENGTH_INV                       | CRC width does not match the<br>amount of user data.<br>Maximum length of the user data<br>from p_c_config.h does not match<br>the configured CRC width. |
|                          | FPAR_VERSION_ERR_INV                      | Invalid F-parameter version                                                                                                                              |
|                          | FPAR_BLOCK_ID_ERR_INV                     | Invalid Block_ID                                                                                                                                         |
|                          | FPAR_CRC1_ERR_INV                         | The CRC of the F-parameters is incor-<br>rect                                                                                                            |
|                          | FPAR_BUILD_NOT_POSSIBLE_IN_DAT<br>AEX_INV | F-parameterization in RUN not possi-<br>ble                                                                                                              |
|                          | FPAR_BUILD_NOT_OK_INV                     | Parameterization inconsistent or<br>HardError situation                                                                                                  |

#### 4.5.8.3 psd\_Config\_Invers(...)

Functionality corresponds to the normal function psd\_Config(...) (Page 34).

|                   |                                            |
|-------------------|--------------------------------------------|
| <b>Function:</b>  | <b>psd_Config_Invers(...)</b>              |
| <b>Prototype:</b> | P_Byte psd_Config_Invers (P_Word Inst_ID); |

| <b>Transfer parameter:</b> | <b>Value range</b>                     | <b>Meaning</b> |
|----------------------------|----------------------------------------|----------------|
| Inst_ID                    | INST_01...INST_X<br>X <= NOF_INSTANCES | Instance ID    |

| <b>Return parameter:</b> | <b>Value range</b>                         | <b>Meaning</b>                      |
|--------------------------|--------------------------------------------|-------------------------------------|
| P_Byte                   | FPAR_CONFIG_OK_INV                         | I/O length configuration successful |
|                          | FPAR_CONFIG_NOT_POSSIBLE_IN_D<br>ATAEX_INV | Configuration in RUN not possible   |
|                          | FPAR_CONFIG_NOT_OK_INV                     | Hard error situation                |

#### 4.5.8.4 psd\_Run\_Invers(...)

Functionality corresponds to the normal function psd\_Run(...) (Page 36).

|                   |                                         |
|-------------------|-----------------------------------------|
| <b>Function:</b>  | <b>psd_Run_Invers(...)</b>              |
| <b>Prototype:</b> | P_Byte psd_Run_Invers (P_Word Inst_ID); |

| <b>Transfer parameter:</b> | <b>Value range</b>                     | <b>Meaning</b> |
|----------------------------|----------------------------------------|----------------|
| Inst_ID                    | INST_01...INST_X<br>X <= NOF_INSTANCES | Instance ID    |

| <b>Return parameter:</b> | <b>Value range</b> | <b>Meaning</b>                                                                                     |
|--------------------------|--------------------|----------------------------------------------------------------------------------------------------|
| P_Byte                   | PSD_DATAEX_INV     | The cyclic data exchange can be started by calling <b>psd_RecvFOutTele_Invers(...)</b> .           |
|                          | PSD_PARAM_INV      | Cyclic data exchange cannot be started. Configuration of the user data lengths missing.            |
|                          | PSD_HARD_FAIL_INV  | Inconsistent instance ID, only in 1-channel HW structure: Inconsistent program flow control value. |

4.5.8.5 **psd\_Stop\_Invers(...)**

Functionality corresponds to the normal function psd\_Stop(...) (Page 37).

|                   |                                                       |
|-------------------|-------------------------------------------------------|
| <b>Function:</b>  | <b>psd_Stop_Invers(...)</b>                           |
| <b>Prototype:</b> | P_Byte psd_Stop (P_Word Inst_ID);                     |
| <b>Meaning:</b>   | Return of the inverse PSD status of the PSD instance. |

| <b>Transfer parameter:</b> | <b>Value range</b>                     | <b>Meaning</b> |
|----------------------------|----------------------------------------|----------------|
| Inst_ID                    | INST_01...INST_X<br>X <= NOF_INSTANCES | Instance ID    |

| <b>Return parameter:</b> | <b>Value range</b> | <b>Meaning</b>                                                                                     |
|--------------------------|--------------------|----------------------------------------------------------------------------------------------------|
| P_Byte                   | PSD_PARAM_INV      | Call from PSD_DATAEX: PSD is parametered and configured                                            |
|                          | PSD_HARD_FAIL_INV  | Inconsistent instance ID, only in 1-channel HW structure: Inconsistent program flow control value. |

4.5.8.6 **psd\_GetFPar\_Invers(...)**

Functionality corresponds to the normal function psd\_GetFPar(...) (Page 64).

|                   |                                            |
|-------------------|--------------------------------------------|
| <b>Function:</b>  | <b>psd_GetFPar_Invers(...)</b>             |
| <b>Prototype:</b> | P_Byte psd_GetFPar_Invers(P_Word Inst_ID); |

| <b>Transfer parameter:</b> | <b>Value range</b>                     | <b>Meaning</b> |
|----------------------------|----------------------------------------|----------------|
| Inst_ID                    | INST_01...INST_X<br>X <= NOF_INSTANCES | Instance ID    |

| <b>Return parameter:</b> | <b>Value range</b>  | <b>Meaning</b>                             |
|--------------------------|---------------------|--------------------------------------------|
| P_Byte                   | GET_FPAR_OK_INV     | Call correct                               |
|                          | GET_FPAR_NOT_OK_INV | Pointer to the F-parameter structure is 0. |

**4.5.8.7 psd\_GetConfig\_Invers(...)**

Functionality corresponds to the normal function psd\_GetConfig(...) (Page 65).

|                   |                                              |
|-------------------|----------------------------------------------|
| <b>Function:</b>  | <b>psd_GetConfig_Invers(...)</b>             |
| <b>Prototype:</b> | P_Byte psd_GetConfig_Invers(P_Word Inst_ID); |

| <b>Transfer parameter:</b> | <b>Value range</b>                     | <b>Meaning</b> |
|----------------------------|----------------------------------------|----------------|
| Inst_ID                    | INST_01...INST_X<br>X <= NOF_INSTANCES | Instance ID    |

| <b>Return parameter:</b> | <b>Value range</b>    | <b>Meaning</b>                                     |
|--------------------------|-----------------------|----------------------------------------------------|
| P_Byte                   | GET_CONFIG_OK_INV     | Call correct                                       |
|                          | GET_CONFIG_NOT_OK_INV | Pointer to configuration parameter structure is 0. |

**4.5.8.8 psd\_GetState\_Invers(...)**

Functionality corresponds to the normal function psd\_GetState(...) (Page 63).

|                   |                                              |
|-------------------|----------------------------------------------|
| <b>Function:</b>  | <b>psd_GetState_Invers(...)</b>              |
| <b>Prototype:</b> | P_Byte psd_GetState_Invers (P_Word Inst_ID); |

| <b>Transfer parameter:</b> | <b>Value range</b>                     | <b>Meaning</b> |
|----------------------------|----------------------------------------|----------------|
| Inst_ID                    | INST_01...INST_X<br>X <= NOF_INSTANCES | Instance ID    |

| <b>Return parameter:</b> | <b>Value range</b> | <b>Meaning</b>                  |
|--------------------------|--------------------|---------------------------------|
| P_Byte                   | PSD_INIT_INV       | Initialized                     |
|                          | PSD_PARAM_INV      | Assigned                        |
|                          | PSD_DATAEX_INV     | Error-free cyclic data exchange |

## Configuration of the PSD

The generation parameters described below are in the PSD file `p_c_config.h`. You need to set all generation parameters.

### 5.1 Number of PSD instances

**#define NOF\_INSTANCES**

Value range: 1 to 32

---

**Note**

The length specification of F-output and F-input user data in `p_c_config.h` is required for each instance used.

---

---

**Note**

Only one instance is possible for 8-bit controllers. Multi-instance operation is not supported for 8-bit controllers.

---

### 5.2 Hardware architecture

**#undef REDUNDANT - 1-channel generation variant**

Use of the PSD at a 1-channel hardware architecture. The PSD stores all the internal data normally and inversely. The PSD functions calculate with the normal and inverse data with different operations/algorithms.

**#define REDUNDANT - 2-channel generation variant**

Use of the PSD at a 2-channel hardware architecture. The processing is carried out on both channels. Data synchronization is carried out through the synchronization interface.

## 5.3 Instance addressing

### **#undef INST\_ACCESS\_POINTER**

With instances >1, addressing of the PSD instance data is made through array index accesses. Depending on the compiler used this can mean a faster access method.

With only one instance, the addressing always made via the array index accesses, wherein the array index "0" is used as a constant.

### **#define INST\_ACCESS\_POINTER**

With instances >1, addressing of the PSD instance data is made through pointer accesses. Depending on the compiler used this can mean a faster access method.

## 5.4 Checking of the source address

### **#undef COMPARE\_F\_SOURCE\_ADD**

The F-source address is not transferred from the F-application to the PSD with the **psd\_FParBuild(...)** function.

### **#define COMPARE\_F\_SOURCE\_ADD**

The F-source address is transferred from the F-application to the PSD with the **psd\_FParBuild(...)** function. The PSD compares it with the F-source address of the F-parameters.

## 5.5 Support of the V1 communication mode

### **#undef V1\_MODE**

The V1 communication mode was defined in the early days of PROFIsafe communication. In the meantime this has been replaced by the V2 communication mode. The V1 mode is not accepted when checking the F-parameters. The cyclic data exchange is thus not possible in V1 mode.

### **#define V1\_MODE**

The V1 mode is accepted when checking the F-parameters.

## 5.6 Selection for using 64 bit arithmetic functions provided compiler-internal or from PSD

### **#undef WITH\_64BIT\_EMULATION**

The compiler used supports the 64-bit data type "unsigned long long". The internal compiler functions for 64-bit arithmetic are used.

### **#define WITH\_64BIT\_EMULATION**

The compiler used does not support the 64-bit data type "unsigned long long". The functions from the p\_64bit.c for 64-bit arithmetic are used.

## 5.7 Integration of fault insertion tests in the PSD OP code

### **#undef FAULT\_INSERTION\_TEST**

Fault-insertion tests are not possible. The error code is deactivated by the compiler switch.

### **#define FAULT\_INSERTION\_TEST**

The error code is activated by the compiler switch. Fault insertion tests can be activated with the **psd\_SetFID(...)** interface function.

### **WARNING**

#### **Requirement: PSD\_RQ\_0\_13**

The fault insertion tests specified in the section Fault insertion test (Page 84) must be performed in accordance with their description. In a release generation, the fault insertion testing must be disabled by disabling the "FAULT\_INSERTION\_TEST" define in p\_c\_config.h when no measures are installed in the F-application to prevent accidental activation of a fault insertion test. The measures (for example, password protection when activating the fault insertion tests, time limitation in the activation process of a fault insertion test, etc.) are to be agreed upon together with the competent licensing authority in accordance with the desired safety level.

## 5.8 Consecutive No. for fault-insertion test activation

### **#define CONS\_NR\_FOR\_FIT\_ACTIVATION value**

A fault insertion test, which has been activated by the **psd\_SetFID(...)** function, is not performed until the PROFIsafe frame has reached the defined consecutive number.

- Value range in V1 mode: 1..0xF0
- Value range in V2 mode: 1..0x00FFFFFF0.

## 5.9 F-input or F-output user data area

```
#define IN- / OUTDATA_MAX_LEN_INST_01 n1
```

```
...
```

```
#define IN- / OUTDATA_MAX_LEN_INST_32 n32
```

You use the defines to set the maximum number of cyclic F-input or F-output data bytes for each instance used. Reconfiguration of the lengths with **psd\_Config(...)** beyond these maximum values is not possible in the PSD\_PARAM state. A "0" means that no F-input or F-output user data are used.

Value range:

- 0 – 12 bytes for 2-byte CRC in V1 mode
- 0 – 12 bytes for 3-byte CRC in V2 mode with F-parameter assignment F\_CRC\_Seed = 0
- 0 – 123 bytes for 4-byte CRC in V1 mode
- 0 – 123 bytes for 4-byte CRC in V2 mode with F-parameter assignment F\_CRC\_Seed = 1

## 5.10 File ID as debug information

```
#define P_CRC
```

```
#define P_INIT
```

```
#define P_INVERS
```

```
#define P_STATE
```

```
#define P_INPUT
```

```
#define P_OUTPUT
```

```
#define P_PARAM
```

```
#define P_TIME
```

The file ID can be used to identify the file called from the **hard\_err(...)** interface function.

## 5.11 Memory attributes

The memory attributes depend on the tool chain used. The key words are to be used correspondingly.

Global variables are defined as follows when using the memory attributes in the PSD:

**Variable type** `P_GLOBDAT_ATTR` **variable name** `P_GLOBDAT_ATTR_POST`;

This allows the assignment of a variable to a memory segment to be made.

**#define** `P_GLOBDAT_ATTR_POINTER`, **#define** `P_GLOBDAT_ATTR_POINTER_POST`

Memory attribute for pointers, used to access to PSD global variables.

**#define** `P_GLOBDAT_ATTR`, **#define** `P_GLOBDAT_ATTR_POST`

Memory attribute for storing the PSD-global variables.

**#define** `P_GLOBDAT_I_ATTR`, **#define** `P_GLOBDAT_I_ATTR_POST`

Memory attribute for storing the PSD-global variables that can be different on both channels.

**#define** `CRC_CONST_ATTR`, **#define** `CRC_CONST_ATTR_POST`

Memory attribute for storing the tables with the lengths of the F-inputs and F-output data and the CRC tables.

## 5.12 Memory model for the CRC tables

This switch effects the activation / deactivation of a memory model for the CRC tables.

The memory attributes depend on the tool chain used. The key words are to be used correspondingly.

**#define** `p_crctab_memory`

The "p\_c\_crc\_m.h" and "p\_c\_crc\_b.h" files are included.

The memory model for the specific tool chain can be set for the CRC table in `p_c_crc_m.h`.

In `p_c_crc_b.h` it is possible to switch back to the memory model for the PSD code.

**#undef** `p_crctab_memory`

The "p\_c\_crc\_m.h" and "p\_c\_crc\_b.h" files are not included.

# Requirements

The F-application must fulfill the requirements summarized below in order to fulfill the demands of the corresponding safety level of the PSD.

## 6.1 Requirements independent of the generation variable

### PSD\_RQ\_0\_01

The normative requirements for HW and FW development and for the implemented HW and FW must be have complied with in accordance with the underlying standards relating to the F-application!

The PSD knows no hardware components at all e.g. memory, microcontroller, timer, I/O interface, power supply. It therefore has no background tests implemented, for example, memory tests, command tests, timer tests.

The PSD has implemented measures to locate the transmission error, error in its program flow and corruption of its variables.

The selection of suitable measures for detecting random hardware errors depends on the system architecture used and lies in the responsibility of the F-slave development.

### PSD\_RQ\_0\_02

You may not change PSD files that are protected with a CRC value.

### PSD\_RQ\_0\_03

The CRC values of the PSD files (individual and overall CRC) have to agree with the values of the CRC check tool.

### PSD\_RQ\_0\_04

Only the PSD interface functions that are declared in the `p_api.h` file may be used.

### PSD\_RQ\_0\_05

Only the PSD interface structures that are declared in the `p_global.h` file may be used.

### PSD\_RQ\_0\_06

The defines from `p_global.h` must be used for the evaluation of the PSD returns.

## PSD\_RQ\_0\_07

Impermissible return values have to induce the **HARD\_FAIL** state of the F-application.  
The following table shows the permitted return values.

| Function                  | Permissible return values                                                                                                                                                                                                                                                                                 |
|---------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| psd_InitInstance(...)     | INIT_INSTANCE_OK                                                                                                                                                                                                                                                                                          |
| psd_FParBuild(...)        | FPAR_BUILD_OK<br>FPAR_F_DEST_ADD_MISMATCH<br>FPAR_F_DEST_ADD_NOT_VALID<br>FPAR_F_SOURCE_ADD_MISMATCH<br>FPAR_F_SOURCE_ADD_NOT_VALID<br>FPAR_F_WD_TIME_NULL<br>FPAR_F_SIL_ERR<br>FPAR_CRC_LENGTH<br>FPAR_VERSION_ERR<br>FPAR_BLOCK_ID_ERR<br>FPAR_CRC1_ERR<br>FPAR_BUILD_NOT_POSSIBLE<br>FPAR_BUILD_NOT_OK |
| psd_Config(...)           | FPAR_CONFIG_OK<br>FPAR_CONFIG_NOT_POSSIBLE                                                                                                                                                                                                                                                                |
| psd_Run(...)              | PSD_DATAEX<br>PSD_PARAM<br>PSD_INIT                                                                                                                                                                                                                                                                       |
| psd_Stop(...)             | PSD_PARAM                                                                                                                                                                                                                                                                                                 |
| psd_RecvFOutTele(...)     | RECV_FOUT_TELE_OK                                                                                                                                                                                                                                                                                         |
| psd_GetFOutData(...)      | F_OUTPUT_OK<br>F_OUTPUT_NOT_OK<br>F_OUTPUT_OLD_CONSNR<br>F_OUTPUT_PASSIVATED<br>F_OUTPUT_COMM_ERR<br>F_OUTPUT_WD_TIMEOUT                                                                                                                                                                                  |
| psd_SetFlnData(...)       | SET_FIN_DATA_OK                                                                                                                                                                                                                                                                                           |
| psd_SendFlnTele(...)      | SEND_FIN_TELE_OK                                                                                                                                                                                                                                                                                          |
| psd_GetState(...)         | PSD_INIT<br>PSD_PARAM<br>PSD_DATAEX                                                                                                                                                                                                                                                                       |
| psd_GetFPar(...)          | GET_FPAR_OK<br>GET_FPAR_NOT_OK                                                                                                                                                                                                                                                                            |
| psd_GetConfig(...)        | GET_CONFIG_OK<br>GET_CONFIG_NOT_OK                                                                                                                                                                                                                                                                        |
| psd_FalsifyFlnTelCRC(...) | FALSIFY_IN_TEL_CRC_OK<br>FALSIFY_IN_TEL_CRC_NOT_OK                                                                                                                                                                                                                                                        |
| psd_SetFID(...)           | FIT_ACTIVATED<br>FIT_INVALID                                                                                                                                                                                                                                                                              |
| psd_GetTraceData(...)     | GET_TRACE_DATA_OK<br>GET_TRACE_DATA_NOT_OK                                                                                                                                                                                                                                                                |

The permissible inverse return value areas of the "inverse" functions (only at 1-channel generation variant) correspond to those of the "normal" functions.

**PSD\_RQ\_0\_08**

The **hard\_err()** function must be implemented by the F-application. It is called by the PSD when it goes into the **PSD\_HARD\_FAIL** state (Fig. 2-4). The function must result in a safe state for all used instances. No further PSD functions need be called within **hard\_err()**.

**PSD\_RQ\_0\_09**

The SIL of the F-parameters may not be greater than the SIL of the F-slave. The SIL of the F-slave must be determined from the F-parameters independent of the SIL.

**PSD\_RQ\_0\_10**

The F-address has to be stored in the F-application. Extraction of the F-address or other values from the F-parameters is not permitted.

**PSD\_RQ\_0\_11**

The F-application must ensure the safe condition for the process for the following return values of **psd\_GetFOutData(...)**:

- F\_OUTPUT\_COMM\_ERR
- F\_OUTPUT\_WD\_TIMEOUT
- F\_OUTPUT\_PASSIVATED
- F\_OUTPUT\_NOT\_OK

**PSD\_RQ\_0\_12**

A PROFIsafe Conformance Test with the qualified layer testing program and the associated test frames must be performed.

**PSD\_RQ\_0\_13**

The fault insertion tests specified in the section Fault insertion test (Page 84) must be performed in accordance with their description. In a release generation, the fault insertion testing must be disabled by disabling the "FAULT\_INSERTION\_TEST" define in **p\_c\_config.h** when no measures are installed in the F-application to prevent accidental activation of a fault insertion test. The measures (for example, password protection when activating the fault insertion tests, time limitation in the activation process of a fault insertion test, etc.) are to be agreed upon together with the competent licensing authority in accordance with the desired safety level.

**PSD\_RQ\_0\_14**

The F-application must set the Status byte.Bit\_1 (Device\_Fault) for at least 2 changes of the monitoring number (2 x the value "F\_OUTPUT\_OK\_NEW\_TEL" in **Telestatus\_2** of the PSD function **psd\_GetFOutData(...)**), if it is unable to respond within the configured timeout period.

6.1 Requirements independent of the generation variable

**PSD\_RQ\_0\_15**

All interface functions except **hard\_err(...)** must return to PSD.

**PSD\_RQ\_0\_16**

The two timer values passed by the F-application to the PSD have to be generated independently of each other (2 timers, 2 quartzes, etc.). The tolerable difference between the two values must be determined and verified by the F-application.

**PSD\_RQ\_0\_17**

The method for addressing an instance is configured (access via pointer or array index). The method has an impact on the run time. The integration test must also include a performance test.

**PSD\_RQ\_0\_18**

The F-application has to take suitable measures in order to recognize different F-output user data or control byte identifiers between the two channels (for example due to an error in the output storage area). This can, for example, occur through comparison of the data transferred by the PSD. The test has to be carried before output to the process.

**PSD\_RQ\_0\_19**

If the PSD signals a communication error, the received control byte is invalid and the control byte identifiers transferred to the F-application CB\_B0\_iParEN, CB\_B1\_OAReq and CB\_B6\_ChF\_Ack are not to be evaluated.

**PSD\_RQ\_0\_20**

If the F-application is in the **HARD\_FAIL** state, the CRC of the F-input frame must be falsified in order for the F-host to faster initiate its measures to achieve the safe state of its application. It does not have to wait for the configured PROFIsafe timeout in this case.

The F-application can use the **psd\_Falsify\_InTelCRC(...)** function for this. No further PSD functions should be called.

## 6.2 Requirements for the 1-channel generation variant

### PSD\_RQ\_1\_01

The return values of the used "inverse" functions are to be evaluated in accordance with the "normal" functions.

### PSD\_RQ\_1\_02

In order to verify the compiler used the analysis is to be carried out with the verification code in the file p\_verify.c.

## 6.3 Requirements for the 2-channel generation variant

### PSD\_RQ\_2\_01

The F-application must provide a synchronization interface with time monitoring for 2-channel PSD generation. Mirroring of the data to be exchanged is to be excluded.

## Test specification

### 7.1 Fault insertion test

The fault insertion tests (FITs) verify the specific implementation of the PSD error detection mechanisms (program execution control, synchronization interface (2 channels), consistency comparison of PSD internal duplicate and inverse data, etc.).

To perform the fault insertion test, in `p_c_config.h`:

- The **"FAULT\_INSERTION\_TEST"** define is to be set
- The consecutive no. with which the test is to be performed with the **"CONS\_NR\_FOR\_FIT\_ACTIVATION"** define must be set. The count of the consecutive no. begins with 0x00FFFFF0 with the first valid PROFIsafe frame.
- The PSD has to be re-generated

The PSD obtains the particular test ID using the **psd\_SetFID(...)** interface function, which is to be called from the F-application after executing **psd\_InitInstance(...)**. Depending on the test a fault is inserted that has to be discovered by the PSD or the F-host.

---

#### Note

Additional fault insertion tests, which are not described here (FIT\_PSD\_004, for example), are performed by the PSD development team and need not be done by you.

---

|                  |                                                                             |
|------------------|-----------------------------------------------------------------------------|
| <b>Function:</b> | <b>psd_SetFID(...)</b>                                                      |
| Prototype:       | P_Byte psd_SetFID (const P_Word Inst_ID,<br>const P_DWord fit_ID_activate); |
| Meaning:         | Transfer of a Test ID to activate the test                                  |

| Transfer parameter: | Value range                                                                                           | Meaning     |
|---------------------|-------------------------------------------------------------------------------------------------------|-------------|
| Inst_ID             | INST_01 ... INST_X<br>X <= NOF_INSTANCES                                                              | Instance ID |
| fit_ID_activate     | The FIT IDs are not defined in <code>p_global.h</code> . They correspond to the FIT names used below. | Test ID     |

| Return parameter: | Value range   | Meaning                            |
|-------------------|---------------|------------------------------------|
| P_Byte            | FIT_ACTIVATED | Test activation successful         |
|                   | FIT_INVALID   | Test ID invalid<br>No test enabled |

During PROFIsafe communication, acceptance criterion must be fulfilled for each test described in the following sections.

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  <b>WARNING</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| <b>Requirement: PSD_RQ_0_13</b><br>The fault insertion tests specified in this section must be performed in accordance with their description. In a release generation, the fault insertion testing must be disabled by disabling the "FAULT_INSERTION_TEST" define in p_c_config.h when no measures are installed in the F-application to prevent accidental activation of a fault insertion test. The measures (for example, password protection when activating the fault insertion tests, time limitation in the activation process of a fault insertion test, etc.) are to be agreed upon together with the competent licensing authority in accordance with the desired safety level. |

### 7.1.1 1-channel and 2-channel generation variant

FIT-ID as well as FIT-Result ID are defined in psd\_fit.h. They correspond to the FIT name used here.

Since Siemens performs additional fault insertion tests, there are gaps in the number range listed here. For example, FIT\_PSD\_004 is missing here in the list. You do not need to perform FITs not listed here.

#### FIT\_PSD\_001

The **psd\_SetFnData(...)** PSD function is passed an instance number that is too large.

##### Conductance of test:

The PROFIsafe communication of the specimen is started, so that the PSD goes to PSD\_DATAEX status. If the consecutive no. reaches the value defined in p\_c\_config.h, the **psd\_SetFnData(...)** function is called with a larger instance no..

##### Acceptance criterion:

The PSD changes to the PSD\_HARD\_FAIL state and calls the Hard Error function of the F-application with the result FIT\_PSD\_RES\_001 in the Line\_No parameter.

## FIT\_PSD\_002

The **psd\_SetFlnData(...)** PSD function is passed a false data length.

### Conductance of test:

The PROFIsafe communication of the specimen is started, so that the PSD goes to PSD\_DATAEX status. If the consecutive no. reaches the value defined in p\_c\_config.h, the **psd\_SetFlnData(...)** function is called with a longer F-user data length.

### Acceptance criterion:

The PSD changes to the PSD\_HARD\_FAIL state and calls the Hard Error function of the F-application.

## FIT\_PSD\_003

The **psd\_SetFlnData(...)** PSD function is not fully executed.

### Conductance of test:

The PROFIsafe communication of the specimen is started, so that the PSD goes to PSD\_DATAEX status. If the consecutive no. reaches the value defined in p\_c\_config.h, the **psd\_SetFlnData(...)** function is terminated early.

### Acceptance criterion:

The PSD changes to the PSD\_HARD\_FAIL state and calls the Hard Error function of the F-application.

## FIT\_PSD\_005

The **psd\_GetInputBuffer(...)** function returns a buffer length too small.

### Conductance of test:

The PROFIsafe communication of the specimen is started, so that the PSD goes to PSD\_DATAEX status. The **psd\_GetInputBuffer(...)** function returns the buffer length for the input frame. If the consecutive no. reaches the value defined in p\_c\_config.h, the returned length is falsified with a value too low.

### Acceptance criterion:

The PSD changes to the PSD\_HARD\_FAIL state and calls the Hard Error function of the F-application with the result FIT\_PSD\_RES\_005 in the Line\_No parameter.

## FIT\_PSD\_006

A CRC error is detected by the F-host.

### Conductance of test:

The PROFIsafe communication of the specimen is started, so that the PSD goes to PSD\_DATAEX status. If the consecutive no. reaches the value defined in p\_c\_config.h, the CRC value in the input frame in the **psd\_SendFInTele(...)** function is falsified.

### Acceptance criterion:

The F-host recognizes a CRC error.

## FIT\_PSD\_011

The **psd\_SendFInTele(...)** function is not fully executed.

### Conductance of test:

The PROFIsafe communication of the specimen is started, so that the PSD goes to PSD\_DATAEX status. If the consecutive no. reaches the value defined in p\_c\_config.h, the **psd\_SendFInTele(...)** function is terminated early.

### Acceptance criterion:

The PSD changes to the PSD\_HARD\_FAIL state and calls the Hard Error function of the F-application with the result FIT\_PSD\_RES\_011 in the Line\_No parameter.

## FIT\_PSD\_013

An incorrect CRC error is recognized in the F-output message frame.

### Conductance of test:

The PROFIsafe communication of the specimen is started, so that the PSD goes to PSD\_DATAEX status. If the consecutive no. reaches the value defined in p\_c\_config.h, the CRC value in the **psd\_GetSIOutData(...)** function is falsified.

### Acceptance criterion:

The PSD detects a communication error and the **psd\_GetFOutData(...)** function returns the return value F\_OUTPUT\_COMM\_ERR back to the F-application.

### FIT\_PSD\_017

The **psd\_RecvFOutTele(...)** function is not fully executed.

#### Conductance of test:

The PROFIsafe communication of the specimen is started, so that the PSD goes to PSD\_DATAEX status. If the consecutive no. reaches the value defined in p\_c\_config.h, the **psd\_RecvFOutTele(...)** function is terminated early.

#### Acceptance criterion:

The PSD changes to the PSD\_HARD\_FAIL state and calls the Hard Error function of the F-application with the result FIT\_PSD\_RES\_017 in the Line\_No parameter.

### FIT\_PSD\_024

The PSD detects a length specification that is too large, which will be passed by the **psd\_GetConfigPtr(...)** function.

#### Conductance of test:

The test can only be performed if the sample uses dynamic length configuration. This is switched on or off in **psd\_InitInstance(...)** by a function parameter.

The length of the F-input data that is passed to the **psd\_GetConfigPtr(...)** function exceeds the length, which has been configured in the length configuration in p\_c\_config.h.

#### Acceptance criterion:

The PSD recognizes the invalid length specification and changes to the PSD\_HARD\_FAIL state. Here it calls the Hard Error function of the F-application with the result FIT\_PSD\_RES\_024 in the Line\_No parameter.

### FIT\_PSD\_025

In the **psd\_FParBuild(...)** function, an F-source address is detected, which does not correspond to the F-source address of the F-application.

#### Conductance of test:

The test can only be performed if the sample uses the Define COMPARE\_F\_SOURCE\_ADD from p\_c\_config.h.

In PSD\_PARAM PSD state, an F-parameter dataset is sent to the F-slave with a falsified F-source address.

#### Acceptance criterion:

The PSD does not accept the F-parameters. A cyclic data exchange with the F-host is not possible.

### 7.1.2 1-channel generation variant

Carry out the following tests:

#### FIT\_PSD\_151

The PSD detects a falsified program flow control value.

**Conductance of test:**

The PROFIsafe communication of the specimen is started, so that the PSD goes to PSD\_DATAEX status. If the consecutive no. reaches the value defined in p\_c\_config.h, the program flow control in the "normal channel" in the **psd\_FlowCtrl(...)** function is falsified.

**Acceptance criterion:**

The PSD changes to the PSD\_HARD\_FAIL state and calls the Hard Error function of the F-application with the result FIT\_PSD\_RES\_151 in the Line\_No parameter.

#### FIT\_PSD\_152

The PSD detects a falsified program flow control value.

**Conductance of test:**

The PROFIsafe communication of the specimen is started, so that the PSD goes to PSD\_DATAEX status. If the consecutive no. reaches the value defined in p\_c\_config.h, the program flow control in the "inverse channel" in the **psd\_FlowCtrl(...)** function is falsified.

**Acceptance criterion:**

The PSD changes to the PSD\_HARD\_FAIL state and calls the Hard Error function of the F-application with the result FIT\_PSD\_RES\_152 in the Line\_No parameter.

### 7.1.3 2-channel generation variant

Carry out the following tests:

#### FIT\_PSD\_102

The **psd\_InTransfer(...)** function, that is called in **psd\_SetFlInData(...)**, returns an incorrect return value.

**Conductance of test:**

The PROFIsafe communication of the specimen is started, so that the PSD goes to PSD\_DATAEX status. If the consecutive no. reaches the value defined in p\_c\_config.h, the return value of the **psd\_InTransfer(...)** function is falsified.

**Acceptance criterion:**

The PSD changes to the PSD\_HARD\_FAIL state and calls the Hard Error function of the F-application with the result FIT\_PSD\_RES\_102 in the Line\_No parameter.

#### FIT\_PSD\_104

The PSD detects a false return value of the **psd\_InSync(...)** function.

**Conductance of test:**

The PROFIsafe communication of the specimen is started, so that the PSD goes to PSD\_DATAEX status. If the consecutive no. reaches the value defined in p\_c\_config.h, the return value of the **psd\_InSync(...)** function is falsified.

**Acceptance criterion:**

The PSD changes to the PSD\_HARD\_FAIL state and calls the Hard Error function of the F-application with the result FIT\_PSD\_RES\_104 in the Line\_No parameter.

#### FIT\_PSD\_106

The PSD detects a false return value of the function **psd\_OutTransfer(...)**.

**Conductance of test:**

The PROFIsafe communication of the specimen is started, so that the PSD goes to PSD\_DATAEX status. If the consecutive no. reaches the value defined in p\_c\_config.h, the return value of the **psd\_OutTransfer(...)** function is falsified.

**Acceptance criterion:**

The PSD changes to the PSD\_HARD\_FAIL state and calls the Hard Error function of the F-application with the result FIT\_PSD\_RES\_106 in the Line\_No parameter.

## FIT\_PSD\_108

The PSD detects a false return value of the **psd\_OutSync(...)** function.

### Conductance of test:

The PROFIsafe communication of the specimen is started, so that the PSD goes to PSD\_DATAEX status. If the consecutive no. reaches the value defined in p\_c\_config.h, the return value of the **psd\_OutSync(...)** function is falsified.

### Acceptance criterion:

The PSD changes to the PSD\_HARD\_FAIL state and calls the Hard Error function of the F-application with the result FIT\_PSD\_RES\_108 in the Line\_No parameter.

## 7.2 Code generation

When code optimization in the code generation, with the 1-channel generation variant ensure that the logical 2-channel capability is maintained in the code.

|                                                                                                                           |
|---------------------------------------------------------------------------------------------------------------------------|
|  <b>WARNING</b>                          |
| <b>Requirement: PSD_RQ_1_02 - 1-channel generation variant</b>                                                            |
| In order to verify the compiler used the analysis is to be carried out with the verification code in the file p_verify.c. |

## 7.3 Compiler verification code for the 1-channel generation variant

The verification code in the **p\_verify.c** file is used to analyze how the employed compiler converts the code sequences used in the PSD.

For proof of the 2-channel capability in the code, all points marked with "M" need to match the assembler instruction described in the verification code .

All positions marked with "S:" show the implementation of C instructions in assembler and are purely informative.

In the comments the term "move" is used for assignments and the term "compare" for comparisons. Theses terms are to be interpreted in accordance with the microcontroller used.

|                                                                                                                           |
|---------------------------------------------------------------------------------------------------------------------------|
|  <b>WARNING</b>                        |
| <b>Requirement: PSD_RQ_1_02</b>                                                                                           |
| In order to verify the compiler used the analysis is to be carried out with the verification code in the file p_verify.c. |

## 7.4 PROFIsafe Conformance-Test

The PROFIsafe Conformance Test is performed using a qualified layer testing program and machine-generated test frames. The test verifies the error-free communication between F-host and F-slave, which results from the PSD functionality and the integration of the PSD in the F-application as well.

The "Layer Tester", which runs on a PC, sends pre-defined PROFIsafe frames via a PROFIBUS or PROFINET map to the F-slave. Subsequently it checks the response message frames against its expectations and stores the result in a file.

The operation of the automated PROFIsafe Conformance Test is described in the document "PROFIsafe Test & Certification related to PROFIsafe V2.6.1" /5/.

The "Layer Tester" including detailed description of the PROFIsafe Conformance Tests can be purchased from the ComDeC (ComDeC@siemens.com).

An accredited testing laboratory (e.g. ComDeC) can be contracted to perform the PROFIsafe Conformance Test and produce an audit report. You can apply for a PROFIsafe certificate for the PNO (<http://www.profibus.com>) with this test report.



### WARNING

#### Requirement: PSD\_RQ\_0\_12

A PROFIsafe Conformance Test with the qualified layer testing program and the associated test frames must be performed.

### Additional information

The PROFIsafe driver V2.2.3 is designed according to the requirements set forth in PI document "PROFIsafe Test Specification" /5/, section 6 "Manufacturer declarations" and is verified for fulfillment of the following properties:

1. Each time the CRC2 calculation results in 0, the result is set to '1'. This rule is described in IEC 61784-3-3 ed.3, section 3.3 "Conventions".
2. The PROFIsafe watchdog is reset only after a change of the (virtual) consecutive number of the F-output frame.

This rule is described in IEC 61784-3-3 ed.3, Table 11: statements "restart device-timer".

3. Transfer failsafe values at the first three PROFIsafe telegram cycles to the F-application independently of the user data from the F-host.

This rule is described in IEC 61784-3-3 ed.3, Table 10: Transition T27: statement "if ok\_nr\_cycles <4, then use FVo, FV\_activated =1".

4. The reception of zero-telegrams is ignored.

If a zero-telegram (all values are 0) is received, the PROFIsafe watchdog continues to run to make sure possible timeouts can be recognized.

This rule is described in IEC 61784-3-3 ed.3, Table 10: statement "ignore safety PDU with all values = 0".

The PROFIsafe driver has been verified by code inspections and module tests for these properties. The verifications and inspections have shown the functional correctness.

## Appendix

### A.1 Assignment of return values to diagnostic messages

The diagnostic messages are specified (Diagnostics section, Table 3) in the PROFIsafe profile 1/. Specific function return values can be assigned to the diagnostic messages:

| PSD function return value                       | Diagnostic alarm no. |
|-------------------------------------------------|----------------------|
| psd_FParBuild(...): FPAR_F_DEST_ADD_MISMATCH    | 0x40                 |
| psd_FParBuild(...): FPAR_F_DEST_ADD_NOT_VALID   | 0x41                 |
| psd_FParBuild(...): FPAR_F_SOURCE_ADD_MISMATCH  | 0x42                 |
| psd_FParBuild(...): FPAR_F_SOURCE_ADD_NOT_VALID | 0x42                 |
| psd_FParBuild(...): FPAR_F_WD_TIME_NULL         | 0x43                 |
| psd_FParBuild(...): FPAR_F_SIL_ERR              | 0x44                 |
| psd_FParBuild(...): FPAR_CRC_LENGTH             | 0x45                 |
| psd_FParBuild(...): FPAR_VERSION_ERR            | 0x46                 |
| psd_FParBuild(...): FPAR_CRC1_ERR               | 0x47                 |
| psd_FParBuild(...): FPAR_BUILD_NOT_OK           | 0x48                 |
| psd_FParBuild(...): FPAR_IPAR_CRC_NOT_VALID     | 0x4B                 |
| psd_FParBuild(...): FPAR_BLOCK_ID_ERR           | 0x4C                 |
| psd_GetFOutData(...): F_OUTPUT_COMM_ERR         | 0x4D                 |
| psd_GetFOutData(...): F_OUTPUT_WD_TIMEOUT       | 0x4E                 |

## A.2 Required space

The memory requirement was determined using the MAP file for generation with the 16-bit Microchip Compiler xc16-gcc v1.33. The information, which also depend on the compiler settings, is used to provide guidance and will vary in accordance with every new PSD generation based on the tool chain used.

### A.2.1 1-channel generation (16-bit controller)

| Compiler switch              | Data<br>[Number of<br>bytes] | Constants<br>[Number of<br>bytes] | Code<br>[Number of<br>bytes] |
|------------------------------|------------------------------|-----------------------------------|------------------------------|
| #define NOF_INSTANCES 1      | 506                          | 4032                              | 43896                        |
| #undef REDUNDANT             |                              |                                   |                              |
| #undef INST_ACCESS_POINTER   |                              |                                   |                              |
| #define COMPARE_F_SOURCE_ADD |                              |                                   |                              |
| #define V1_MODE              |                              |                                   |                              |
| #undef WITH_64BIT_EMULATION  |                              |                                   |                              |

### A.2.2 2-channel generation (16-bit controller)

| Compiler switch              | Data<br>[Number of<br>bytes] | Constants<br>[Number of<br>bytes] | Code<br>[Number of<br>bytes] |
|------------------------------|------------------------------|-----------------------------------|------------------------------|
| #define NOF_INSTANCES 1      | 314                          | 3936                              | 25239                        |
| #def REDUNDANT               |                              |                                   |                              |
| #undef INST_ACCESS_POINTER   |                              |                                   |                              |
| #define COMPARE_F_SOURCE_ADD |                              |                                   |                              |
| #define V1_MODE              |                              |                                   |                              |
| #undef WITH_64BIT_EMULATION  |                              |                                   |                              |

## A.3 Static code analysis

The tool "PC-lint for C/C++ V9.00g" was used for the static code analysis. The standard settings of the au-sm123.lnt and au-misra3.lnt MISRA modules were used. A code analysis did not provide any errors or warnings.

## Index of abbreviations / terms

|                                        |                                                                                                                                                                                                                                                       |
|----------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| CRC                                    | Cyclic Redundancy Check                                                                                                                                                                                                                               |
| CRC1                                   | F-parameter CRC                                                                                                                                                                                                                                       |
| CRC2                                   | Input/output frame CRC                                                                                                                                                                                                                                |
| CRC3                                   | CRC iParameter                                                                                                                                                                                                                                        |
| F                                      | Fail-safe                                                                                                                                                                                                                                             |
| F-Host                                 | Data processing unit, which is able to execute the PROFIsafe protocol and request data exchange with the F-slave.<br>See also the definition in the PROFIsafe Profile, section 3.                                                                     |
| FIT                                    | Fault Insertion Test                                                                                                                                                                                                                                  |
| F-parameter                            | PROFIsafe parameter                                                                                                                                                                                                                                   |
| F-slave                                | Communication partner that is able to execute the PROFIsafe protocol and whose data exchange is initiated by the F-host.<br>See also the definition in the PROFIsafe Profile, section 3.                                                              |
| F-submodule                            | Fail-safe submodule                                                                                                                                                                                                                                   |
| FW                                     | Firmware                                                                                                                                                                                                                                              |
| HW                                     | Hardware                                                                                                                                                                                                                                              |
| iParameter                             | Module-specific parameters                                                                                                                                                                                                                            |
| ISR                                    | Interrupt Service Routine                                                                                                                                                                                                                             |
| NULL frame                             | All bytes of a frame contain "0".                                                                                                                                                                                                                     |
| PNO                                    | PROFIBUS-Nutzer-Organisation (PROFIBUS user organization)<br>Additional information can be found on the Internet ( <a href="http://www.profibus.com/">http://www.profibus.com/</a> ).                                                                 |
| PROFIBUS DP                            | Process Field Bus Distributed Peripherals                                                                                                                                                                                                             |
| PROFINET IO                            | Process Field Network Input - Output                                                                                                                                                                                                                  |
| PROFIsafe trailer                      | Composed of status/control byte and the security information (CRC2).                                                                                                                                                                                  |
| PSD                                    | PROFIsafe Driver for F-Slaves/F-Submodules                                                                                                                                                                                                            |
| SI                                     | Stack Interface                                                                                                                                                                                                                                       |
| SIL                                    | Safety Integrity Level                                                                                                                                                                                                                                |
| SW                                     | Software                                                                                                                                                                                                                                              |
| WD                                     | Watchdog for runtime monitoring                                                                                                                                                                                                                       |
| "HARD_FAIL" state of the F-application | The F-application has detected an inconsistency, probably as a result of a hardware fault. During the transition to this state, the F-application sets the safe state (for example, its specific safety values in the input and output process data). |

## Bibliography

|     |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
|-----|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| /1/ | <p>PROFIsafe – Profile for Safety Technology on PROFIBUS and PROFINET IO<br/>Version 2.6MU1, August 2018</p> <p>Order no. 3.192</p> <p>Publisher: PROFIBUS User Organization e.V.<br/>Haid-und-Neu-Str. 7, D-76131 Karlsruhe</p> <p>Additional information can be found on the Internet (<a href="http://www.profibus.com">http://www.profibus.com</a>).</p>                                                                                                                          |
| /2/ | <p>IEC 61508:2010<br/>Functional Safety of electrical/electronic/programmable electronic safety-related systems</p> <p>Part 1: General requirements<br/>Part 2: Requirements for electrical/electronic/programmable electronic safety-related systems<br/>Part 3: Software requirements</p>                                                                                                                                                                                           |
| /3/ | <p>PROFIsafe starter kit</p> <p>Article number 6ES7195-3BF03-0YA0</p> <p>Additional information can be found on the Internet<br/>(<a href="https://support.industry.siemens.com/cs/products/6es7195-3bf03-0ya0/profisafer-starterkit-v3-5-1?pid=752385&amp;mlfb=6ES7195-3BF03-0YA0&amp;mfn=ps&amp;lc=en-US">https://support.industry.siemens.com/cs/products/6es7195-3bf03-0ya0/profisafer-starterkit-v3-5-1?pid=752385&amp;mlfb=6ES7195-3BF03-0YA0&amp;mfn=ps&amp;lc=en-US</a>).</p> |
| /4/ | <p>PROFIsafe Environmental Requirements<br/>Version 2.6 – Date: December 2015</p> <p>Order no. 3.242</p> <p>Publisher: PROFIBUS User Organization e.V.<br/>Haid-und-Neu-Str. 7, D-76131 Karlsruhe</p> <p>Additional information can be found on the Internet (<a href="http://www.profibus.com">http://www.profibus.com</a>).</p>                                                                                                                                                     |
| /5/ | <p>PROFIsafe Test Specification (Psafe-Testspec_2242_V23_Mar18.pdf)</p> <p>Order no. 2.242</p> <p>Publisher: PROFIBUS User Organization e.V.<br/>Haid-und-Neu-Str. 7, D-76131 Karlsruhe</p> <p>Additional information can be found on the Internet (<a href="http://www.profibus.com">http://www.profibus.com</a>).</p>                                                                                                                                                               |

|     |                                                                                                                                                                                                                                                                                                               |
|-----|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| /6/ | <p>PROFINET Security Guideline<br/>Version 2.0, Nov. 2013</p> <p>Order no. 7.002</p> <p>Publisher: PROFIBUS User Organization e.V.<br/>Haid-und-Neu-Str. 7, D-76131 Karlsruhe</p> <p>Additional information can be found on the Internet (<a href="http://www.profibus.com">http://www.profibus.com</a>).</p> |
| /7/ | <p>Readme.pdf<br/>Version 1.1, January 2019</p> <p>Publisher: SIEMENS AG<br/>Provided in the product package of the PROFIsafe Starter Kit.</p>                                                                                                                                                                |
| /8/ | <p>Remote IO for Factory Automation Common Profile Specification for PROFINET and PROFIBUS<br/>Version 1.10 – Date: August 2018</p> <p>Order no. 3.242</p> <p>Publisher: PROFIBUS User Organization e.V.<br/>Haid-und-Neu-Str. 7, D-76131 Karlsruhe</p>                                                       |